

Variational Autoencoder in Single-cell Data

Yongjin Park

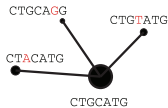
10 March 2026

Learning objectives

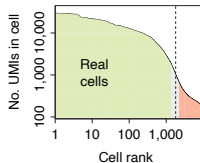
- Model-based approach to high-dimensional data analysis
- Primer to learning with artificial neural networks

Overview of single-cell data analysis

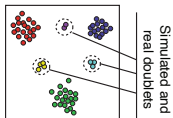
Alignment and molecular counting



Cell filtering and quality control



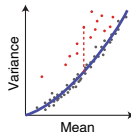
Doublet scoring



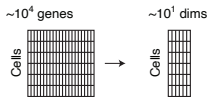
Cell size estimation

Cells	c_1	c_2	c_3
Gene ₁	2	4	20
Gene ₂	1	2	10
Gene ₃	3	6	30
Cell depth:	6	12	60

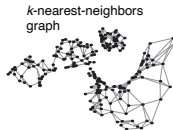
Gene variance analysis



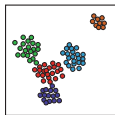
Reduction to a medium-dimensional space



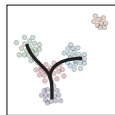
Manifold representation



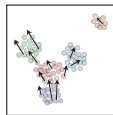
Clustering and differential expression



Trajectories



Velocity estimation



(Kharchenko 2021)

The goal of modelling in high-dimensional space

The goal is to model two types of relationships:

- Relationship between dimensions/features (genes)
- Relationship between samples/data points (cells)

Key question

Which cells co-vary? Which genes co-vary?

What is a latent variable/factor model?

Examples:

- Principal Component Analysis
- Non-negative matrix factorization
- (...)

What is Unsupervised Learning?

Supervised learning

- Input: data $\mathcal{X} = \{\mathbf{x}_i, \dots\}$
- Input 2: label $\mathcal{Y} = \{y_i, \dots\}$
- Goal: learn $f : \mathcal{X} \rightarrow \mathcal{Y}$

X: gene expression samples; Y: disease labels

Unsupervised learning

- Input: data $\mathcal{X} = \{\mathbf{x}_i, \dots\}$
- Goal 1: learn $f : \mathcal{Z} \rightarrow \mathcal{X}$
- Goal 2: hidden/latent states, $\mathcal{Z} = \{\mathbf{z}_i, \dots\}$

X: gene expression matrices

Today's lecture

- 1 Representation Learning
- 2 MNIST Example
- 3 Building Autoencoders on Single-Cell Data
- 4 Variational Autoencoder on single cell data
- 5 Multinomial likelihood

How can we obtain a good representation of data?

Two strategies to learn a “good” representation

1. Feature engineering

Construct useful combinations of variables, feature engineering

- Principal component analysis (matrix factorization)
- Pattern recognition
- Clustering (average patterns $\approx$ combined features/data)

2. Manifold learning

Change the manifold (a coordinate system) of data

- Probabilistic topic models: word frequency $\rightarrow$ simplex

We use both strategies in practice.

Two strategies to learn a “good” representation

1. Feature engineering

Construct useful combinations of variables, feature engineering

- Principal component analysis (matrix factorization)
- Pattern recognition
- Clustering (average patterns $\approx$ combined features/data)

2. Manifold learning

Change the manifold (a coordinate system) of data

- Probabilistic topic models: word frequency $\rightarrow$ simplex
- Angular space $\rightarrow$ Euclidean space

We use both strategies in practice.

Two strategies to learn a “good” representation

1. Feature engineering

Construct useful combinations of variables, feature engineering

- Principal component analysis (matrix factorization)
- Pattern recognition
- Clustering (average patterns $\approx$ combined features/data)

2. Manifold learning

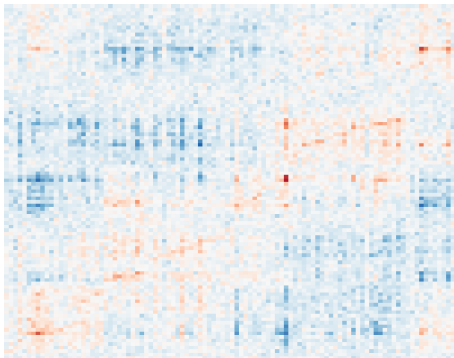
Change the manifold (a coordinate system) of data

- Probabilistic topic models: word frequency $\rightarrow$ simplex
- Angular space $\rightarrow$ Euclidean space
- Observed high-dim. $\rightarrow$ interpretable ones

We use both strategies in practice.

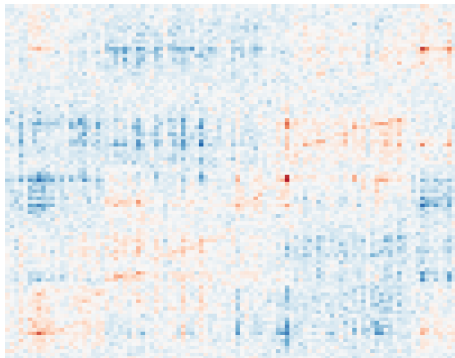
Feature engineering by combining multiple vars.

$$\mathbf{x}_i = \mathbf{u}_i V^\top + \mathcal{N}(\mathbf{0}, I)$$

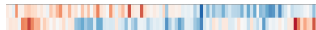


Feature engineering by combining multiple vars.

$$\mathbf{x}_i = \mathbf{u}_i V^\top + \mathcal{N}(\mathbf{0}, I)$$



features



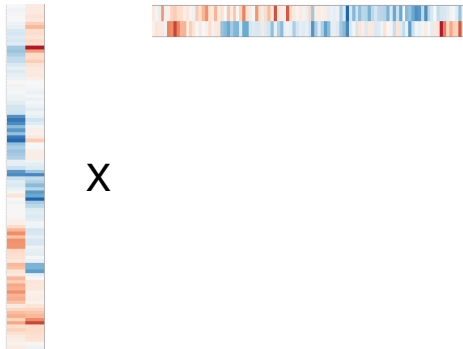
feature loading

X

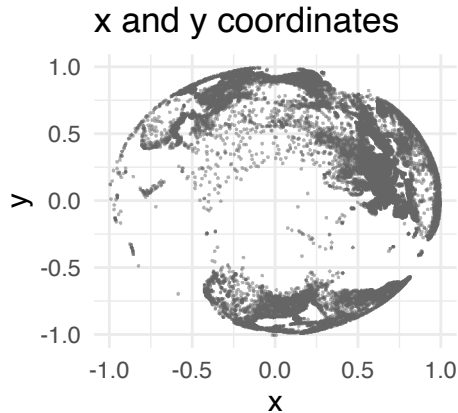
Feature engineering

$$\mathbf{x}_i = \mathbf{u}_i V^\top + \mathcal{N}(\mathbf{0}, I)$$

- We call each column of U and V a factor (row/column)
- One factor corresponds to the activities/patterns/expressions of 100 genes/samples
- We will discuss further later.



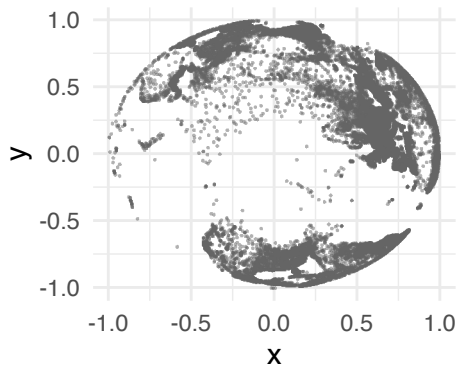
Manifold learning changes the coordinate system



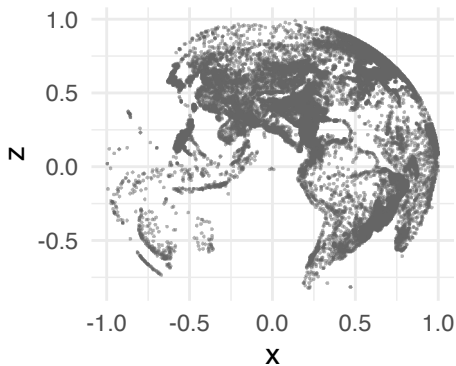
Data from <https://simplemaps.com/data/world-cities>

Manifold learning changes the coordinate system

x and y coordinates

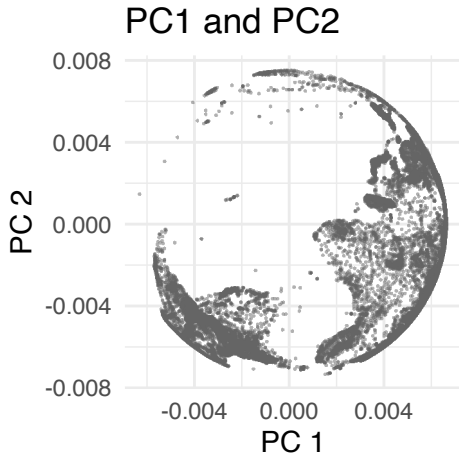


x and z coordinates



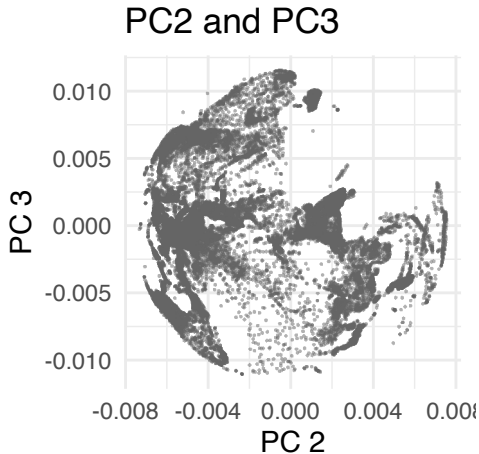
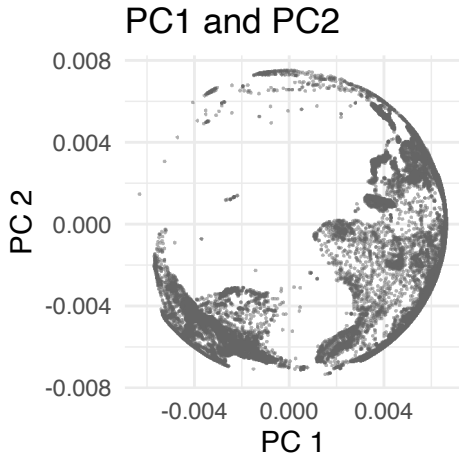
Data from <https://simplemaps.com/data/world-cities>

Manifold learning changes the coordinate system



Data from <https://simplemaps.com/data/world-cities>

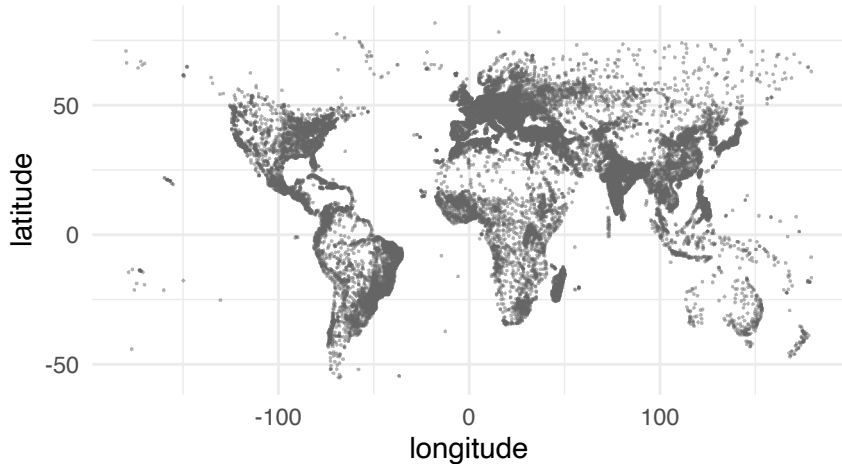
Manifold learning changes the coordinate system



Data from <https://simplemaps.com/data/world-cities>

Manifold learning changes the coordinate system

(x,y,z) to (long, lat)



Representation learning

Feature engineering

- Combine samples based on similarity to learn common features
- Combine features/genes/proteins to create a new factor
- Matrix factorization to capture factors that explain large variation

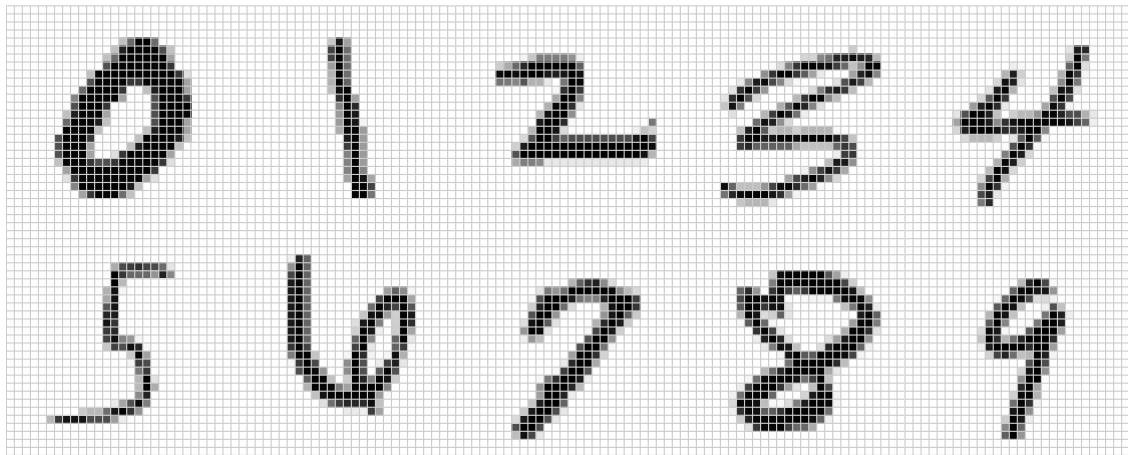
Manifold learning

- Change the coordinate system
- What's not separable in one space can be spread out in another one
- Apply non-linear functions to a set of features

Today's lecture

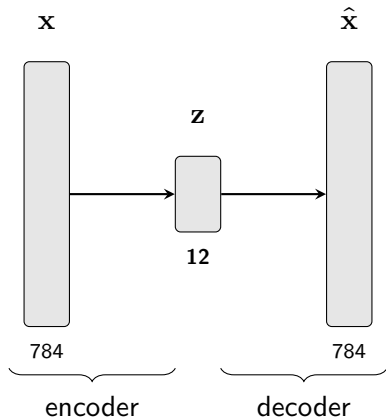
- 1 Representation Learning
- 2 MNIST Example**
- 3 Building Autoencoders on Single-Cell Data
- 4 Variational Autoencoder on single cell data
- 5 Multinomial likelihood

A visual example: handwritten digits



- $28 \times 28 = 784$ pixels per image — can we compress to $K = 12$?
- We train on all digits (0-9)

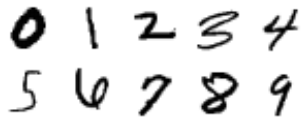
The autoencoder bottleneck $\approx$ PCA



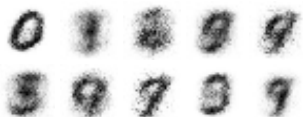
- All information about the digit must pass through **12 numbers**

AE vs. VAE on all digits (K=12)

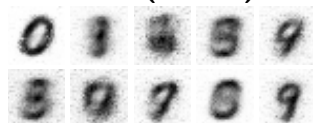
Original



AE (K=12)

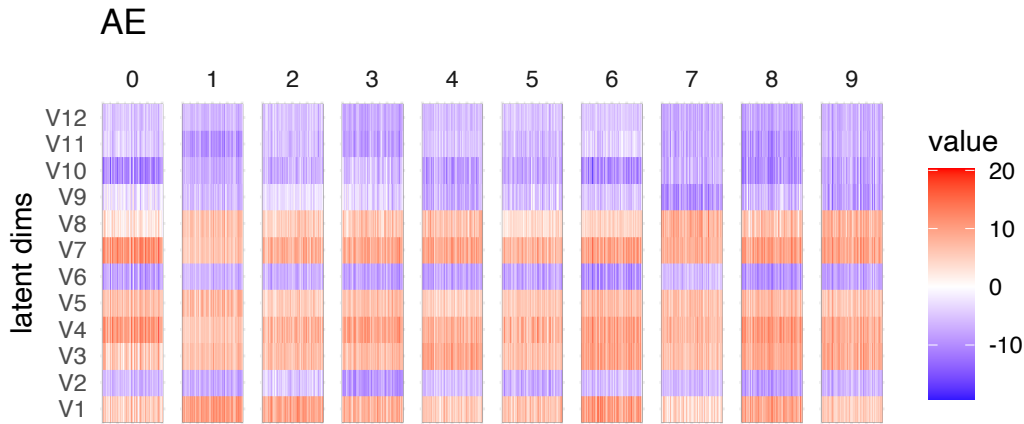


VAE (K=12)



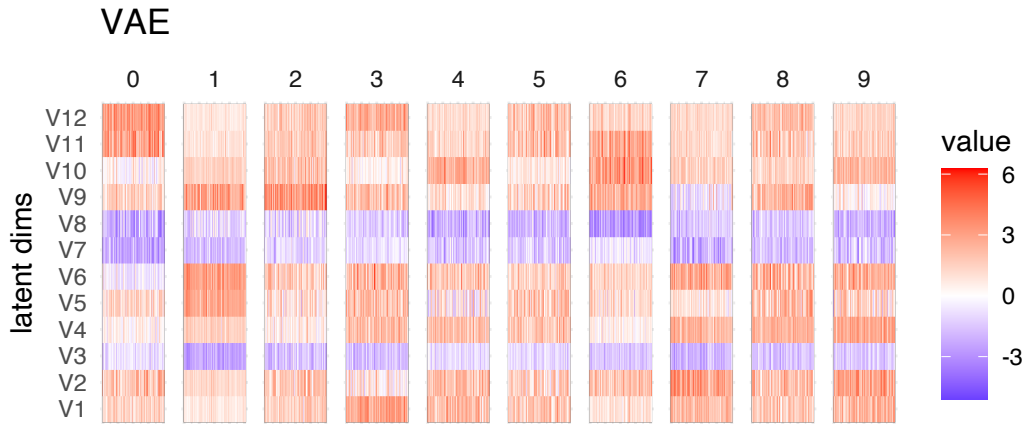
- Trained on all digits (0-9)

Latent representations (heatmaps sorted by digit)



- What differences do you observe between AE and VAE?
- How are the digits distributed in latent space?

Latent representations (heatmaps sorted by digit)



- What differences do you observe between AE and VAE?
- How are the digits distributed in latent space?

Why non-linearity?

- **Linear layers alone** cannot learn complex patterns
- Stacking linear layers: $f(g(\mathbf{x})) = \mathbf{W}_2(\mathbf{W}_1\mathbf{x}) = (\mathbf{W}_2\mathbf{W}_1)\mathbf{x}$
- Result is still linear — just a single matrix multiplication

ReLU activation

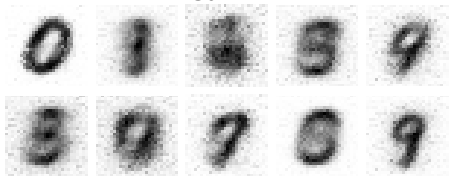
- **ReLU:** $\text{ReLU}(z) = \max(0, z)$
- Introduces non-linearity between layers
- Simple, computationally efficient, avoids vanishing gradients

Two-layer VAE architecture

- $\mathbf{x} \xrightarrow{\text{Linear}} \mathbf{h} \xrightarrow{\text{ReLU}} \mathbf{h}' \xrightarrow{\text{Linear}} \mathbf{z}$
- Hidden layer $\mathbf{h}$ (128 dimensions) learns intermediate features
- Batch normalization stabilizes training

Nonlinear layers VAE results

Linear VAE

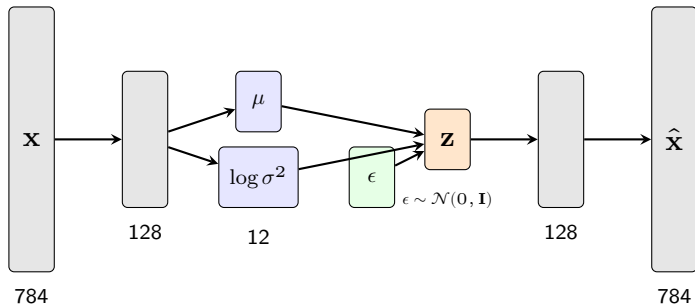


Nonlinear layers VAE



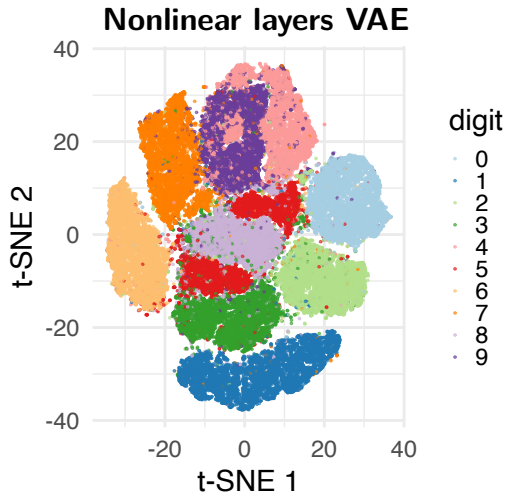
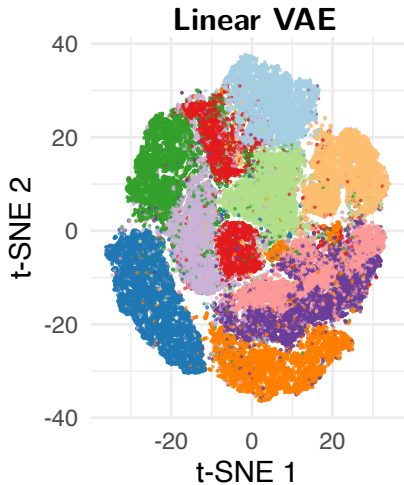
- 2 encoder layers: $784 \rightarrow 128 \rightarrow 12$ (with batch norm + ReLU)
- 2 decoder layers: $12 \rightarrow 128 \rightarrow 784$ (with batch norm + ReLU)

Nonlinear layers VAE architecture



- **Reparameterization trick:** $\mathbf{z} = \mu + \sigma \odot \epsilon$ where $\epsilon \sim \mathcal{N}(0, \mathbf{I})$

t-SNE visualization of latent space



- t-SNE reduces $K=12$ dimensions to 2D for visualization

How do we learn nonlinear encoding-decoding?

- Encoder: $\mathbf{z} \leftarrow f(\cdots f(\mathbf{x}))$
- Decoder: $\hat{\mathbf{x}} \leftarrow g(\cdots g(\mathbf{z}))$
- Loss: $\mathcal{L} = L(\mathbf{x}, \hat{\mathbf{x}})$
- Gradient (differentiation): $\nabla \mathcal{L}$

Today's lecture

- 1 Representation Learning
- 2 MNIST Example
- 3 Building Autoencoders on Single-Cell Data**
- 4 Variational Autoencoder on single cell data
- 5 Multinomial likelihood

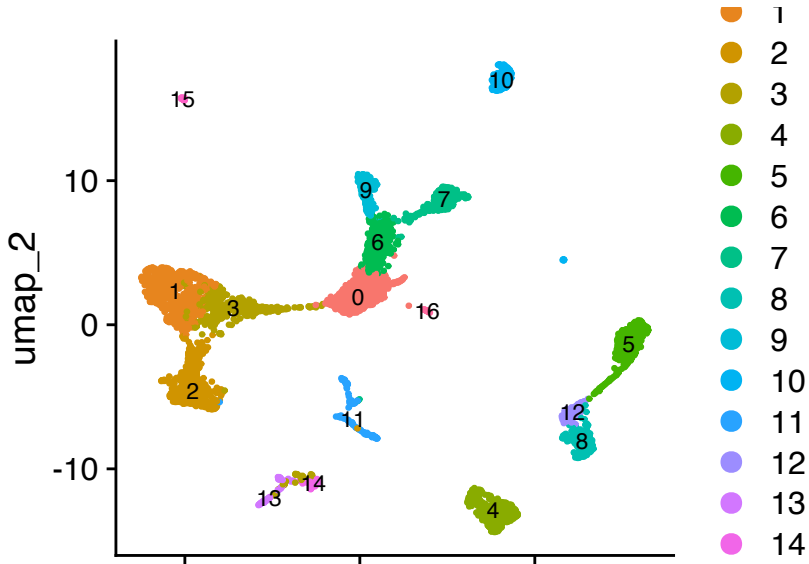
The PDAC data

```
## Load and merge tissues 1-3
pdac.list <- lapply(1:3, function(i) {
  Read10X(data.dir = paste0("PDAC_TISSUE_", i, "/")) |>
  CreateSeuratObject(project = paste0("PDAC_T", i))
})
pdac <- merge(pdac.list[[1]], y = c(pdac.list[[2]], pdac.list[[3]]))

## Process
pdac <- pdac |>
  NormalizeData() |>
  FindVariableFeatures(nfeatures = 2000) |>
  ScaleData() |>
  RunPCA(npcs = 30) |>
  FindNeighbors(dims = 1:20) |>
  FindClusters(resolution = 0.5) |>
  RunUMAP(dims = 1:20)
```

- 21,611 genes/features/rows and 4,469 cells/columns

PDAC: 4469 cells from 3 tissues



Prepare the data for torch

```
counts <- GetAssayData(pdac, layer = "counts")
hvg <- VariableFeatures(pdac)
X <- t(as.matrix(counts[hvg, ]))
```

- Extract **raw counts** (not normalized/batch-corrected)
- Rows = cells, columns = genes (2000 highly variable genes)

```
dim(torch.data$X)
```

```
## [1] 4469 2000
```

What does the data look like?

```
torch.data$X[1:5, 1:6]
```

```
##
##      CLPS IGJ PNLIP SPP1 AMY2A RARRES1
## T1_AAACGAAAGTGGAAAG-1    0  0    0    0    0    0
## T1_AAAGAACCATTAAAGG-1    0  0    0  249    0    2
## T1_AAAGGATTCGGCTTGG-1    0  0    0    0    0    1
## T1_AAAGGGCAGTAGCAAT-1    0  0    0  123    0    1
## T1_AAAGGTACACATACGT-1    0  0    0   12    0    0
```

- Sparse count matrix – most entries are zero

```
mean(torch.data$X == 0)
```

```
## [1] 0.9054329
```

Convert to a torch tensor

```
X.tensor <- torch_tensor(torch.data$X)
X.tensor$shape
```

```
## [1] 4469 2000
```

- This is our input: each row $\mathbf{x}_i$ is one cell's expression profile

What is an autoencoder?

- Compress high-dimensional data into a low-dimensional **code**
- Then reconstruct the original data from that code

$$\mathbf{x} \xrightarrow{\text{encoder}} \mathbf{z} \xrightarrow{\text{decoder}} \hat{\mathbf{x}}$$

- The **bottleneck** $\mathbf{z}$ forces the model to learn a compact summary

A linear encoder

$$\mathbf{z}_i = W \mathbf{x}_i + \mathbf{b}$$

- Input: $\mathbf{x}_i \in \mathbb{R}^p$ (2000 genes)
- Output: $\mathbf{z}_i \in \mathbb{R}^K$ (e.g., $K = 10$ latent dimensions)

```
K <- 10

linear.encoder <- nn_module(
  initialize = function(d.in, K) {
    self$linear <- nn_linear(d.in, K)
  },
  forward = function(x) {
    x |> torch_log1p() |> self$linear()
  }
)
```

A linear decoder

$$\hat{\mathbf{x}}_i = W' \mathbf{z}_i + \mathbf{b}'$$

- Input: $\mathbf{z}_i \in \mathbb{R}^K$
- Output: $\hat{\mathbf{x}}_i \in \mathbb{R}^p$ (reconstructed expression)

```
linear.decoder <- nn_module(  
  initialize = function(K, d.out) {  
    self$linear <- nn_linear(K, d.out)  
  },  
  forward = function(z) {  
    self$linear(z)  
  }  
)
```

Putting them together

```
autoencoder <- nn_module(  
  initialize = function(d.in, K) {  
    self$enc <- linear.encoder(d.in, K)  
    self$dec <- linear.decoder(K, d.in)  
  },  
  forward = function(x) {  
    z <- self$enc(x)  
    x.hat <- self$dec(z)  
    list(z = z, x.hat = x.hat)  
  }  
)
```

- Encoder compresses $p \rightarrow K$, decoder expands $K \rightarrow p$

Loss function: mean squared error

$$\mathcal{L} = \frac{1}{n} \sum_i \|\mathbf{x}_i - \hat{\mathbf{x}}_i\|^2$$

- How well can we reconstruct the original data?

```
recon.loss <- function(x, x.hat) {  
  torch_mean(torch_sum((x - x.hat)^2, dim = 2))  
}
```

Training: set up the model and optimizer

```
set.seed(42)
model.ae <- autoencoder(n.genes, K)
optimizer <- optim_adam(model.ae$parameters, lr = 1e-3)
```

```
model.ae
```

```
## An `nn_module` containing 42,010 parameters.
```

```
##
```

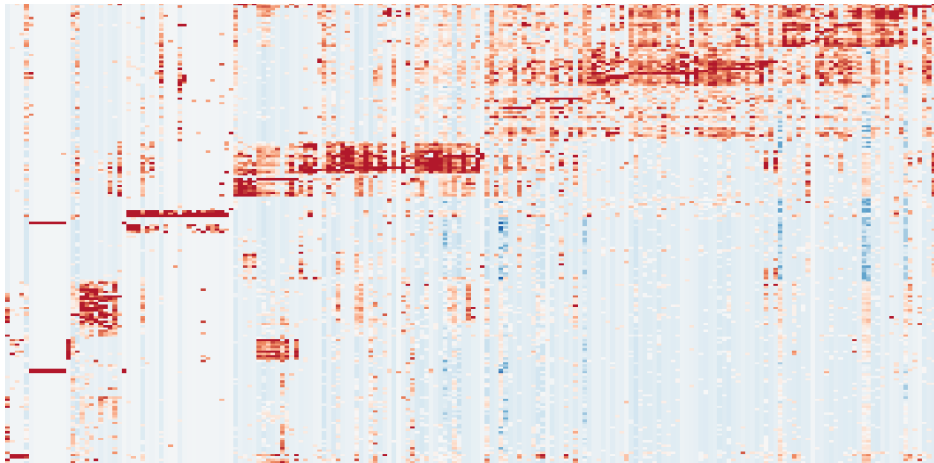
```
## -- Modules -----
```

```
## * enc: <nn_module> #20,010 parameters
```

```
## * dec: <nn_module> #22,000 parameters
```

First, let's sort the observed data for visualization

Observed (log1p, scaled) — 200 cells x 200 genes

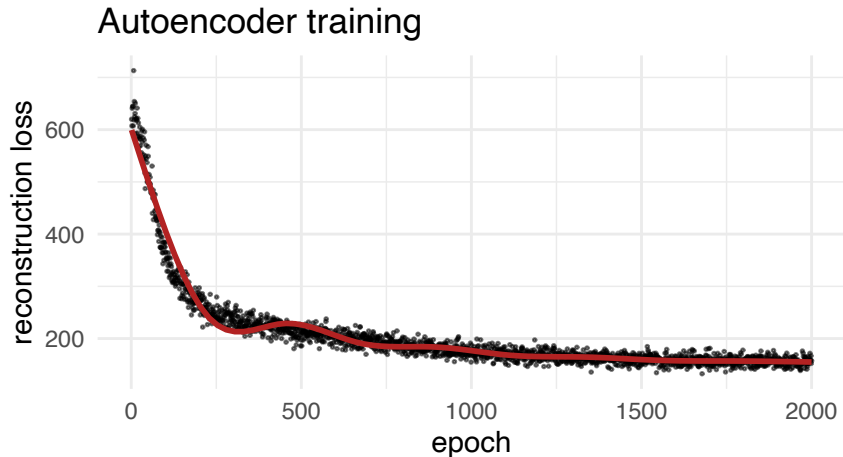


Training loop

```
n.epoch <- 2000
batch.size <- 256
for (epoch in seq_len(n.epoch)) {
  idx <- sample(n.cells, batch.size)
  x.batch <- X.tensor[idx, ]
  out <- model.ae(x.batch)
  loss <- recon.loss(torch_log1p(x.batch), out$x.hat)
  optimizer$zero_grad()
  loss$backward()
  optimizer$step()
}
```

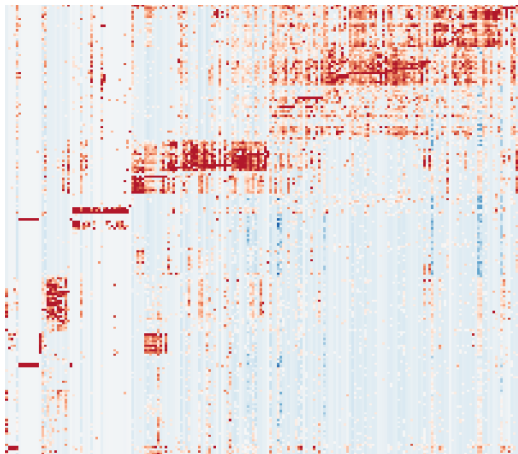
- Sample a minibatch → forward pass → compute loss → backprop → update weights

Training loss

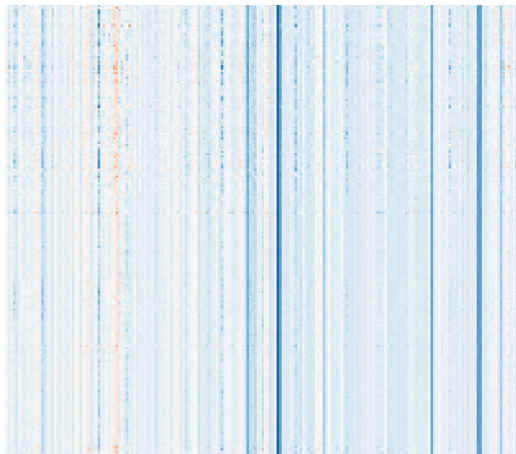


Reconstruction across epochs

Observed

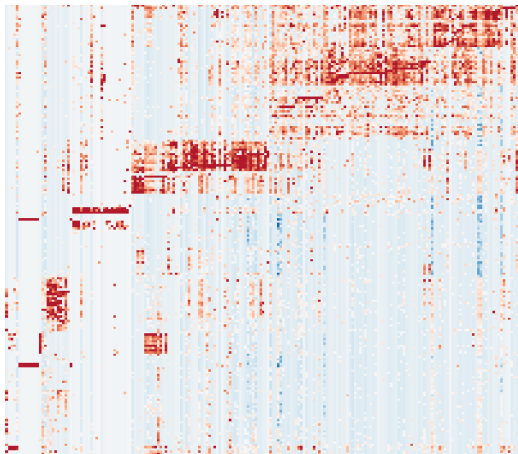


epoch 1

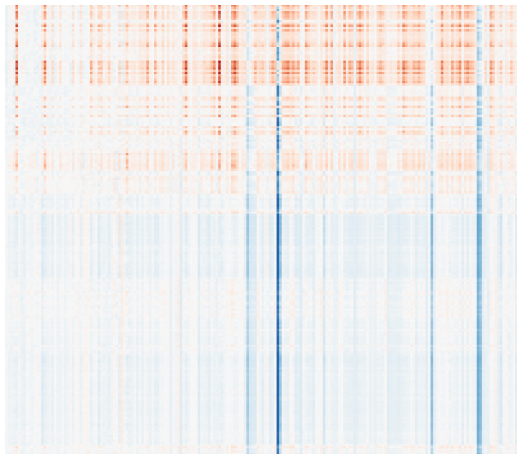


Reconstruction across epochs

Observed

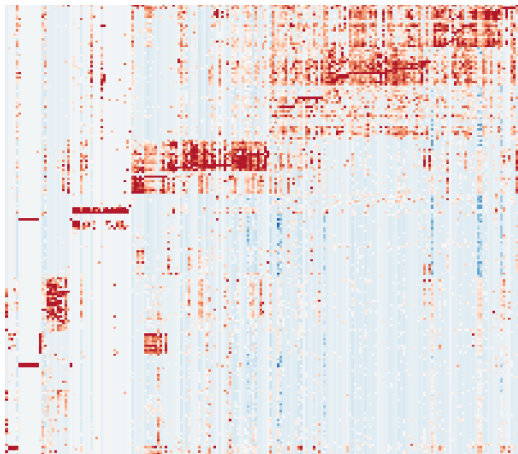


epoch 100

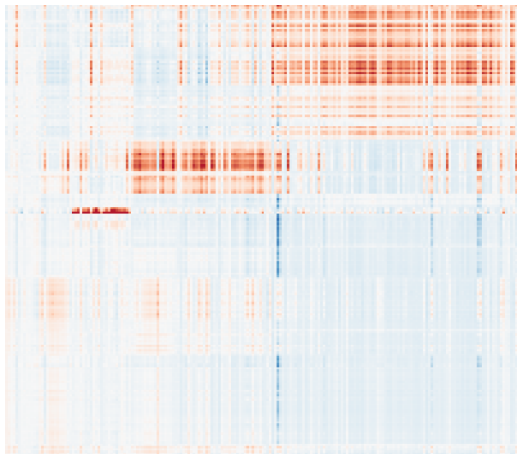


Reconstruction across epochs

Observed

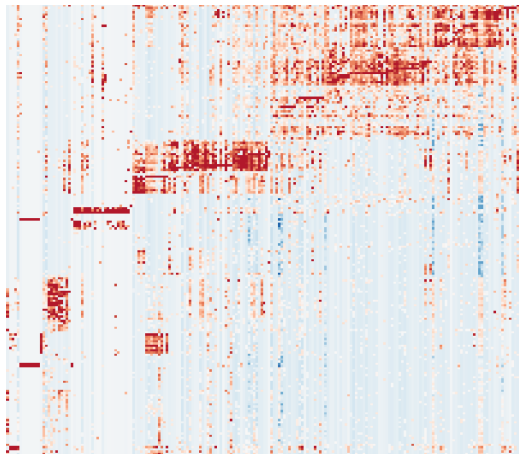


epoch 500

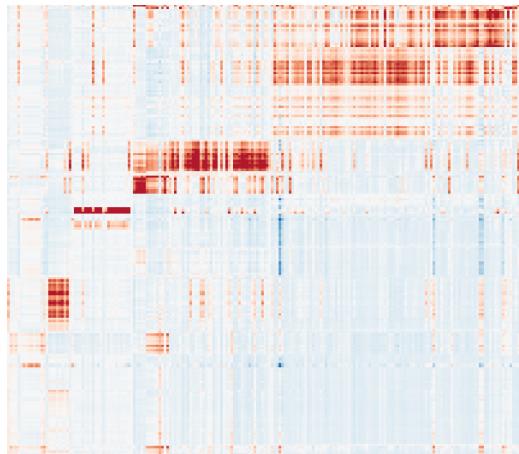


Observed vs. reconstructed (epoch 2000)

Observed

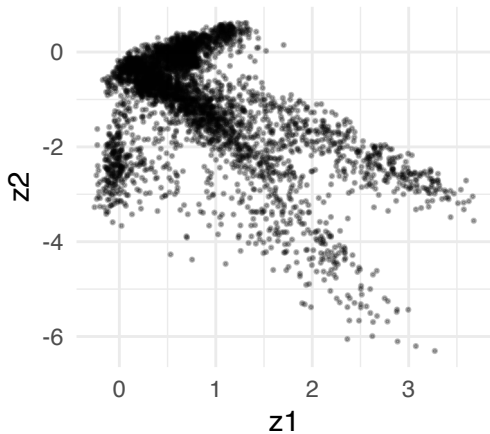


Reconstructed (epoch 2000)

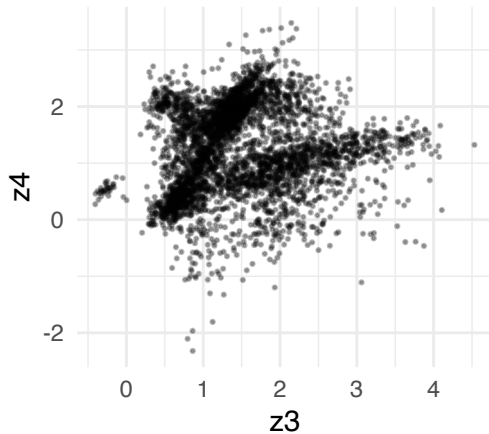


Latent space of the autoencoder

z1 vs z2



z3 vs z4



- Each dot is a cell; the latent space has no particular structure yet

Today's lecture

- 1 Representation Learning
- 2 MNIST Example
- 3 Building Autoencoders on Single-Cell Data
- 4 Variational Autoencoder on single cell data**
- 5 Multinomial likelihood

From autoencoder to VAE: one key change

Autoencoder encoder outputs a **point**:

$$\mathbf{z}_i = W\mathbf{x}_i + \mathbf{b}$$

VAE encoder outputs a **distribution**:

$$\mu_i = W_\mu \mathbf{x}_i + \mathbf{b}_\mu, \quad \log \sigma_i^2 = W_\sigma \mathbf{x}_i + \mathbf{b}_\sigma$$

$$\mathbf{z}_i \sim \mathcal{N}(\mu_i, \text{diag}(\sigma_i^2))$$

The reparameterization trick

We can't backpropagate through a random sample. But we can rewrite:

$$\mathbf{z}_i = \mu_i + \sigma_i \odot \epsilon, \quad \epsilon \sim \mathcal{N}(\mathbf{0}, I)$$

- The randomness is in ϵ (external), not in the parameters
- Gradients flow through μ and σ as usual

VAE encoder in torch

```
vae.encoder <- nn_module(  
  initialize = function(d.in, K, h = 128) {  
    self$hidden <- nn_linear(d.in, h)  
    self$bn <- nn_batch_norm1d(h)  
    self$z.mean <- nn_linear(h, K)  
    self$z.logvar <- nn_linear(h, K)  
  },  
  forward = function(x) {  
    x <- torch_log1p(x)  
    h <- self$hidden(x) |> self$bn() |> nnf_relu()  
    mu <- self$z.mean(h)  
    lv <- self$z.logvar(h) |> torch_clamp(-8, 8)  
    ## reparameterization trick  
    z <- mu + torch_exp(lv / 2) * torch_randn_like(lv)  
    list(z = z, mu = mu, logvar = lv)  
  }  
)
```

What changed? Only the encoder

Autoencoder encoder

```
forward = function(x) {  
  x |>  
    torch_log1p() |>  
    self$linear()  
}
```

Returns a **point** z

VAE encoder

```
forward = function(x) {  
  x <- torch_log1p(x)  
  h <- self$hidden(x) |>  
    self$bn() |> nnf_relu()  
  mu <- self$z.mean(h)  
  lv <- self$z.logvar(h) |>  
    torch_clamp(-8, 8)  
  z <- mu + torch_exp(lv/2) *  
    torch_randn_like(lv)  
  list(z=z, mu=mu, logvar=lv)  
}
```

Returns **z sampled**

KL divergence: regularize the latent space

$$D_{\text{KL}}(q(\mathbf{z}|\mathbf{x}) \parallel p(\mathbf{z})) = -\frac{1}{2} \sum_k (1 + \log \sigma_k^2 - \mu_k^2 - \sigma_k^2)$$

- Pushes each $q(\mathbf{z}|\mathbf{x})$ toward the prior $\mathcal{N}(\mathbf{0}, I)$
- Without it, the encoder would set $\sigma \rightarrow 0$ (killing the noise)

```
kl.loss <- function(mu, logvar) {  
  -0.5 * torch_mean(  
    torch_sum(1 + logvar - mu^2 - torch_exp(logvar), dim = 2)  
  )  
}
```

VAE = encoder + decoder + new loss

```
vae <- nn_module(  
  initialize = function(d.in, K) {  
    self$enc <- vae.encoder(d.in, K)  
    self$dec <- linear.decoder(K, d.in)  
  },  
  forward = function(x) {  
    q <- self$enc(x)  
    x.hat <- self$dec(q$z)  
    list(x.hat = x.hat, mu = q$mu,  
         logvar = q$logvar, z = q$z)  
  }  
)
```

- Same linear decoder as before
- Loss = reconstruction + KL divergence

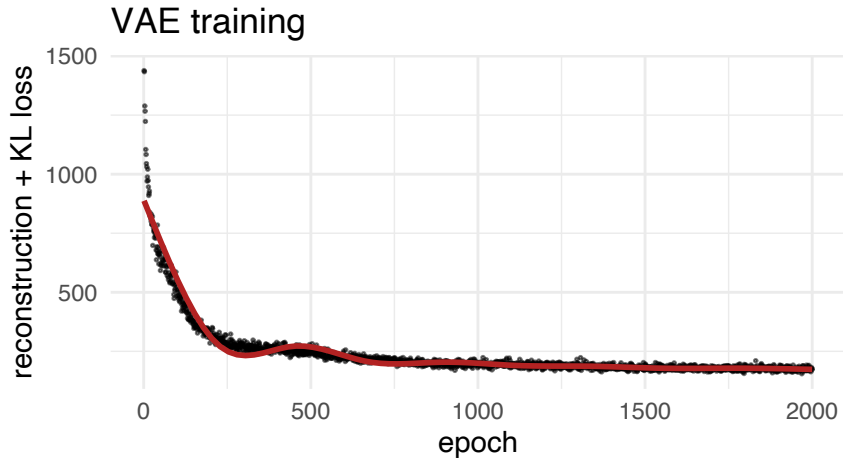
Train the VAE

```
model.vae <- vae(n.genes, K)
optimizer <- optim_adam(model.vae$parameters, lr = 1e-3)

for (epoch in seq_len(n.epoch)) {
  idx <- sample(n.cells, batch.size)
  x.batch <- X.tensor[idx, ]
  out <- model.vae(x.batch)
  loss <- recon.loss(torch_log1p(x.batch), out$x.hat) +
    kl.loss(out$mu, out$logvar)
  optimizer$zero_grad()
  loss$backward()
  optimizer$step()
}
```

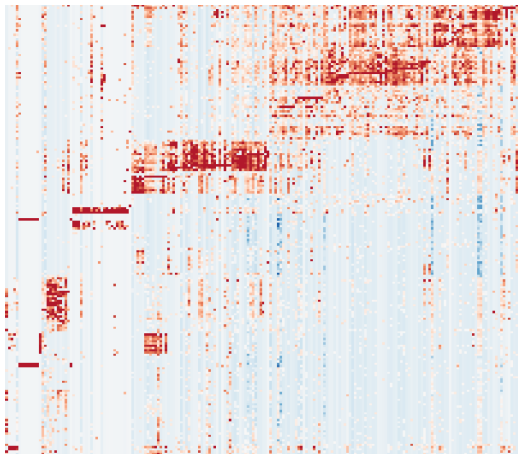
- One extra term in the loss: that's it

VAE training loss

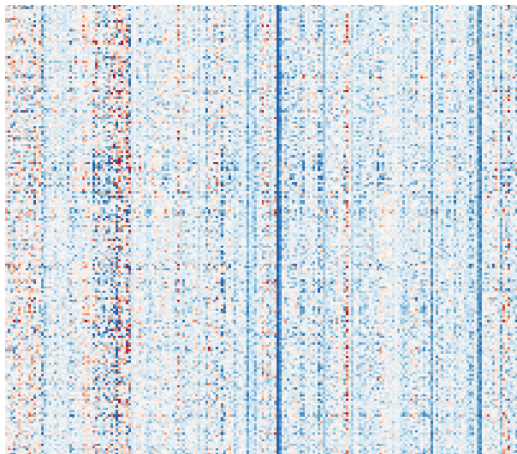


VAE reconstruction across epochs

Observed

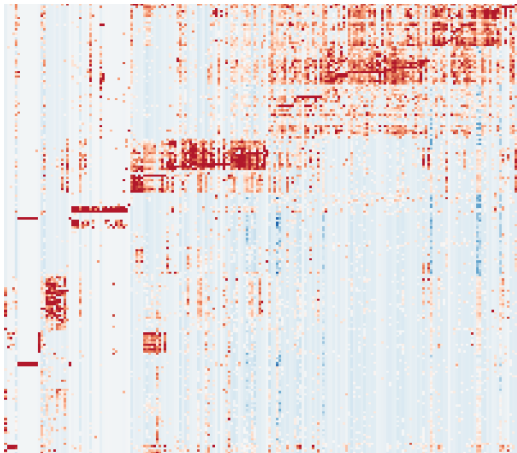


epoch 1

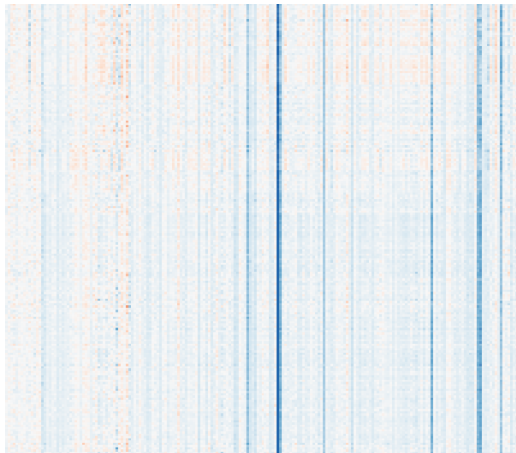


VAE reconstruction across epochs

Observed

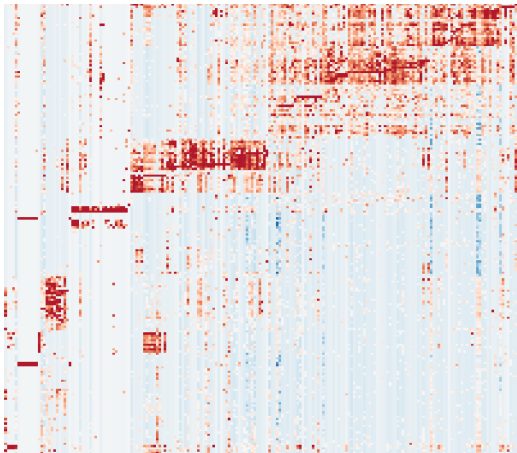


epoch 100

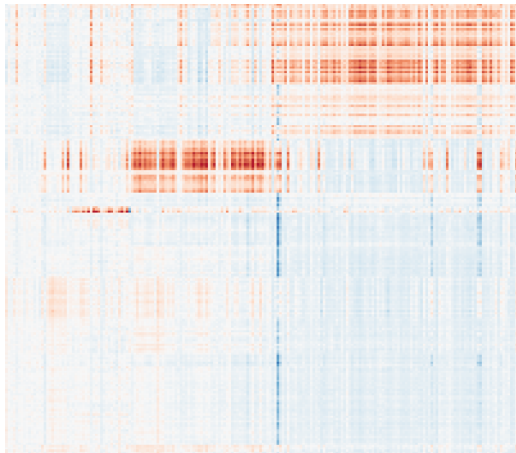


VAE reconstruction across epochs

Observed

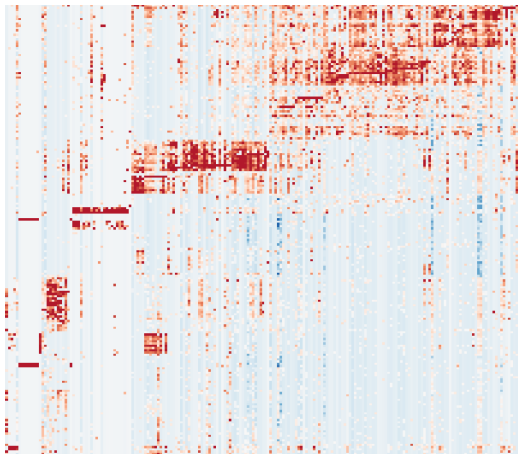


epoch 500

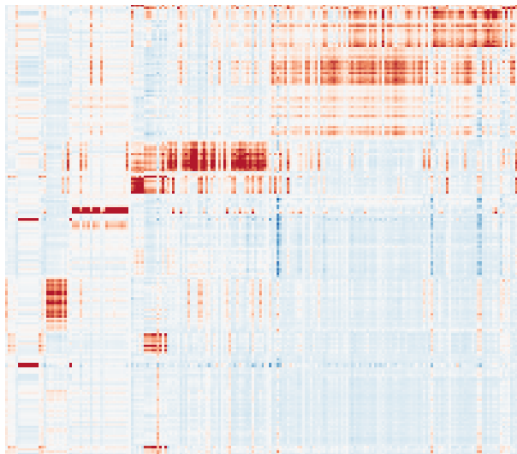


VAE: observed vs. reconstructed (epoch 2000)

Observed

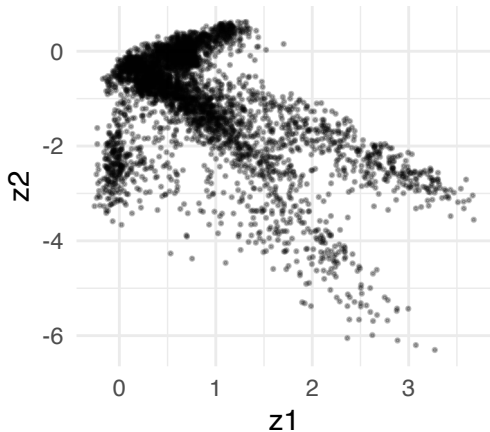


VAE (epoch 2000)

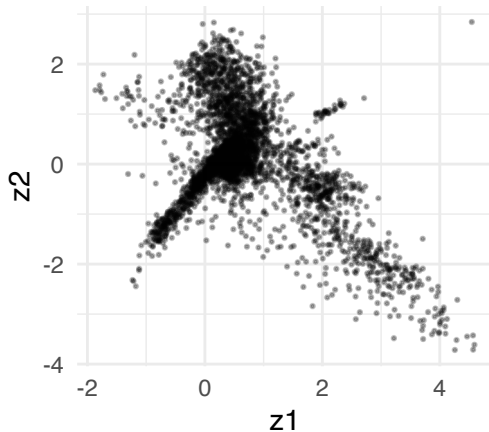


Latent space: autoencoder vs. VAE

Autoencoder: z1 vs z2



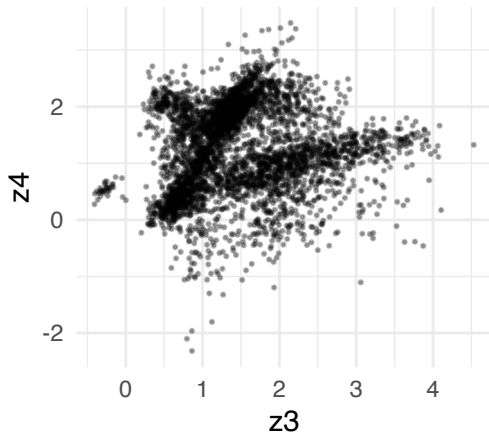
VAE: z1 vs z2



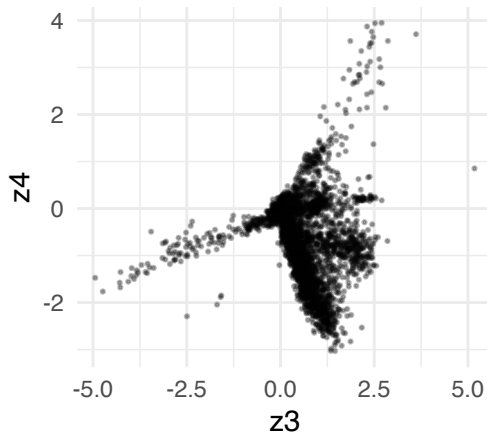
- Each dot is a cell; the latent space has no particular structure yet
- ▲ VAE latent space is more structured (KL pushes toward $\mathcal{N}(0, I)$)

Latent space: autoencoder vs. VAE

Autoencoder: z3 vs z4



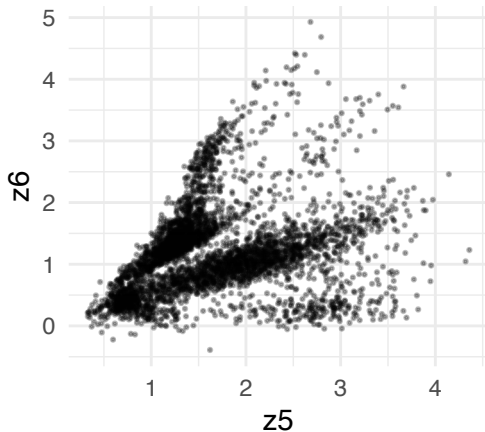
VAE: z3 vs z4



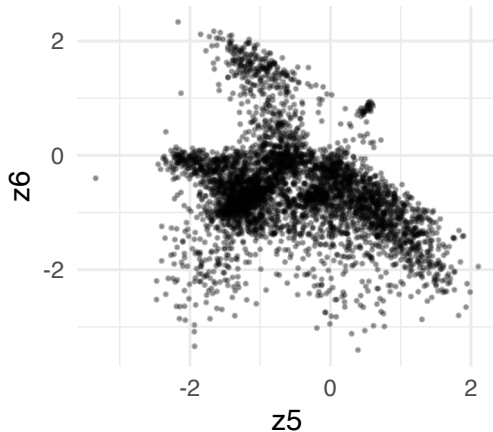
- Each dot is a cell; the latent space has no particular structure yet
- ▲ VAE latent space is more structured (KL pushes toward $\mathcal{N}(0, I)$)

Latent space: autoencoder vs. VAE

Autoencoder: z5 vs z6



VAE: z5 vs z6



- Each dot is a cell; the latent space has no particular structure yet
- ▲ VAE latent space is more structured (KL pushes toward $\mathcal{N}(0, I)$)

Today's lecture

- 1 Representation Learning
- 2 MNIST Example
- 3 Building Autoencoders on Single-Cell Data
- 4 Variational Autoencoder on single cell data
- 5 Multinomial likelihood**

The problem with MSE loss

Our data is **counts**, not continuous values

- MSE on $\log(1 + x)$ ignores the count nature of the data
- A cell with 10,000 total reads and a cell with 1,000 reads are treated the same

Better: model each cell's expression as a **multinomial** draw

$$\mathbf{x}_i \sim \text{Multinomial}(N_i, \pi_i)$$

- $N_i = \sum_j x_{ij}$ is the library size
- π_i is the gene probability vector

Multinomial decoder

Each cell i has **topic proportions** θ_i and each topic k has **gene probabilities** ϕ_k :

$$\theta_i = \text{softmax}(\mathbf{z}_i), \quad \phi_k = \text{softmax}(\beta_k)$$

$$\pi_i = \theta_i^\top \Phi = \sum_k \theta_{ik} \phi_k$$

- This is exactly a **topic model**: cells are documents, genes are words

Multinomial decoder in torch

```
multinom.decoder <- nn_module(  
  initialize = function(K, d.out) {  
    self$logit.beta <- nn_parameter(  
      torch_randn(K, d.out)  
    )  
  },  
  forward = function(z) {  
    log.theta <- nnf_log_softmax(z, dim = 2)  
    log.phi <- nnf_log_softmax(self$logit.beta, dim = 2)  
    ## log(theta %*% phi)  
    log.pi <- torch_logsumexp(  
      log.theta$unsqueeze(3) + log.phi$unsqueeze(1),  
      dim = 2  
    )  
    log.pi  
  },  
  get_weight = function() {  
    self$logit.beta  
  }  
)
```

Multinomial log-likelihood

$$\log p(\mathbf{x}_i \mid \pi_i) = \sum_j x_{ij} \log \pi_{ij} + \text{const.}$$

```
multinom.llik <- function(x, log.pi) {  
  torch_mean(torch_sum(x * log.pi, dim = 2))  
}
```

- We maximize the log-likelihood (minimize negative log-likelihood)
- No need for $\log(1 + x)$ transform — we work directly with counts

Autoencoder with multinomial likelihood

```
ae.multinom <- nn_module(  
  initialize = function(d.in, K) {  
    self$enc <- linear.encoder(d.in, K)  
    self$dec <- multinom.decoder(K, d.in)  
  },  
  forward = function(x) {  
    z <- self$enc(x)  
    log.pi <- self$dec(z)  
    list(z = z, log.pi = log.pi)  
  }  
)
```

VAE with multinomial likelihood

```
vae.multinom <- nn_module(  
  initialize = function(d.in, K) {  
    self$enc <- vae.encoder(d.in, K)  
    self$dec <- multinom.decoder(K, d.in)  
  },  
  forward = function(x) {  
    q <- self$enc(x)  
    log.pi <- self$dec(q$z)  
    list(log.pi = log.pi, mu = q$mu,  
         logvar = q$logvar, z = q$z)  
  }  
)
```

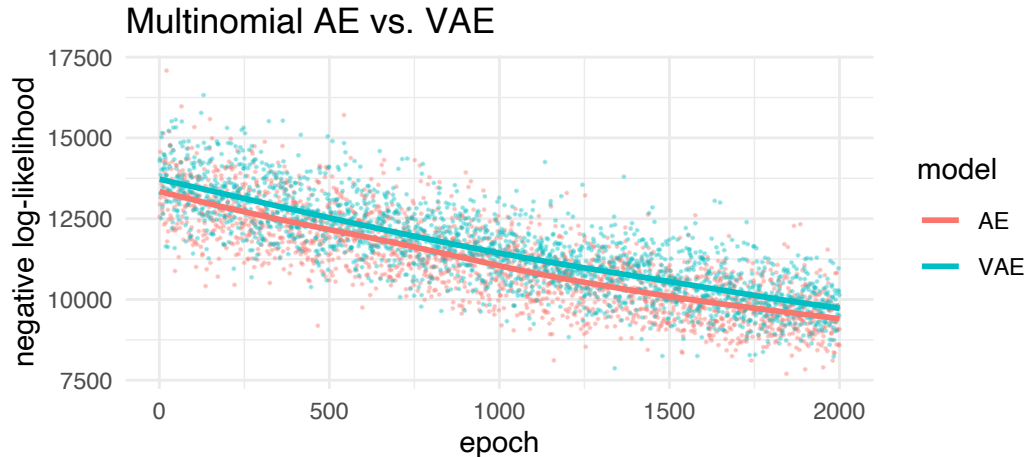
- Same encoder changes as before (point $\rightarrow$ distribution)
- Now the decoder is shared: multinomial in both cases

Train both models

```
## AE: minimize negative log-likelihood  
loss <- -multinom.llik(x.batch, out$log.pi)  
  
## VAE: add KL divergence  
loss <- -multinom.llik(x.batch, out$log.pi) +  
        kl.loss(out$mu, out$logvar)
```

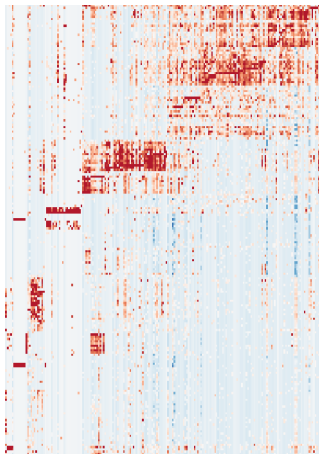
- The only change from MSE: swap the reconstruction loss

Multinomial training loss

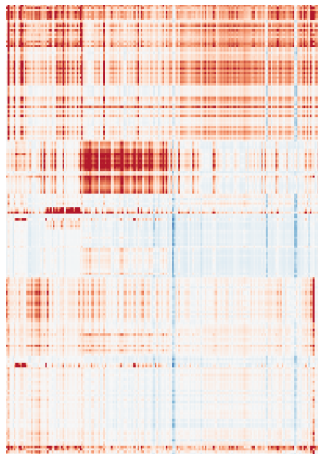


Multinomial reconstruction (AE vs. VAE, epoch 2000)

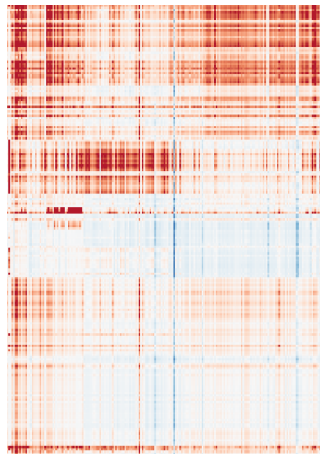
Observed



AE (multinomial)

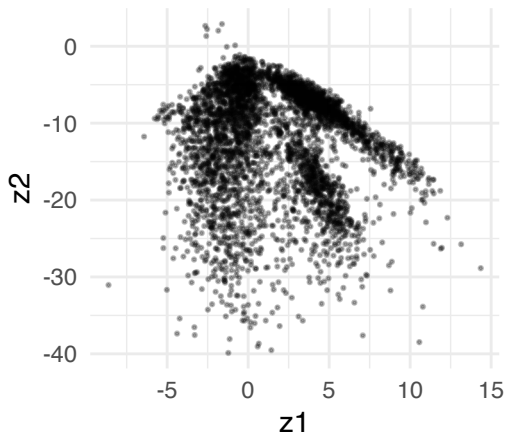


VAE (multinomial)

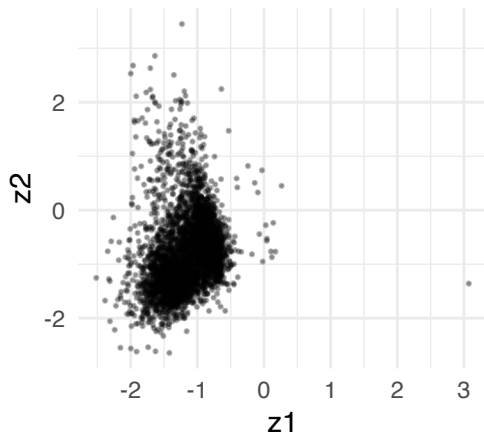


Latent space: multinomial AE vs. VAE

AE (multinomial): z1 vs z2

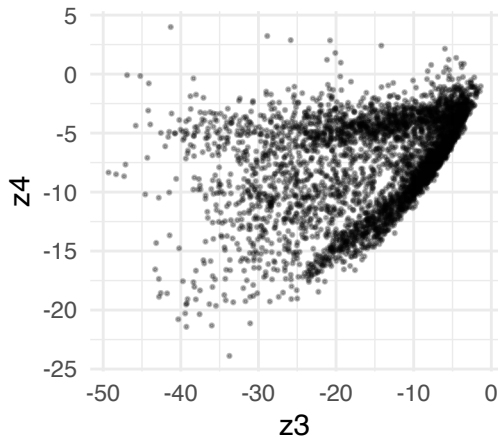


VAE (multinomial): z1 vs z2

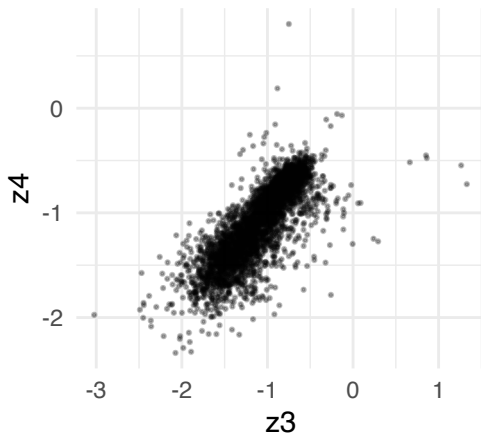


Latent space: multinomial AE vs. VAE

AE (multinomial): z3 vs z4

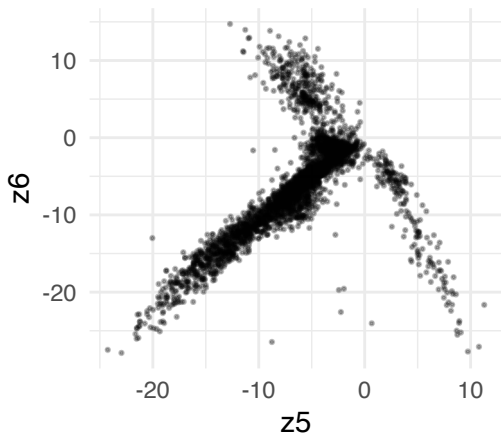


VAE (multinomial): z3 vs z4

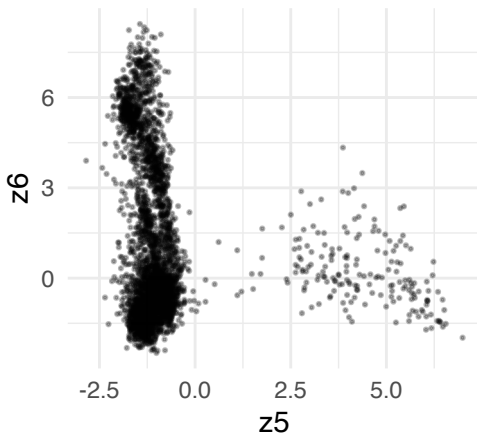


Latent space: multinomial AE vs. VAE

AE (multinomial): z5 vs z6



VAE (multinomial): z5 vs z6



Decoder structure: top genes per factor

Without the KL term, the encoder can “cheat”:

- Set $\sigma \rightarrow 0$ — no noise, just a point estimate
- The latent space becomes **irregular** — no structure
- Some dimensions may **collapse** (carry no information)

β -**VAE** (Higgins et al. 2016): weight the KL term to control the tradeoff

$$\mathcal{L} = -\log p(\mathbf{x}|\mathbf{z}) + \beta \cdot D_{\text{KL}}(q(\mathbf{z}|\mathbf{x})\|p(\mathbf{z}))$$

- $\beta = 0$: pure autoencoder (no regularization)
- $\beta = 1$: standard VAE
- $\beta > 1$: stronger push toward $\mathcal{N}(0, I)$

Training with different β values

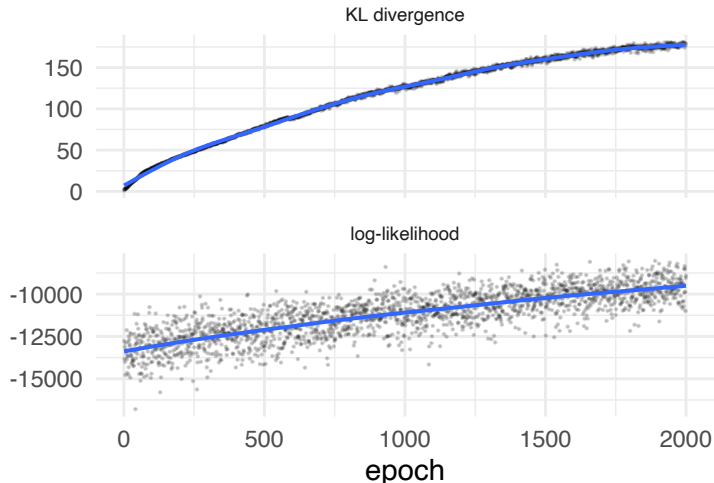
```
betas <- c(0, 0.1, 0.5, 1, 2, 5)

for (beta in betas) {
  ## ... same training loop, but:
  loss <- -multinom.llik(x, out$log.pi) +
    beta * kl.loss(out$mu, out$logvar)
}
```

- We track KL and log-likelihood **separately** at each epoch

Loss traces across epochs

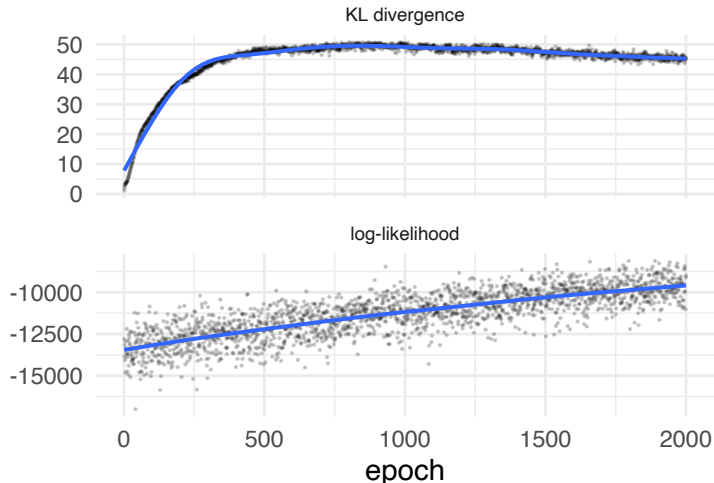
$\beta = 0$



- Higher β : lower KL (more Gaussian) but worse reconstruction

Loss traces across epochs

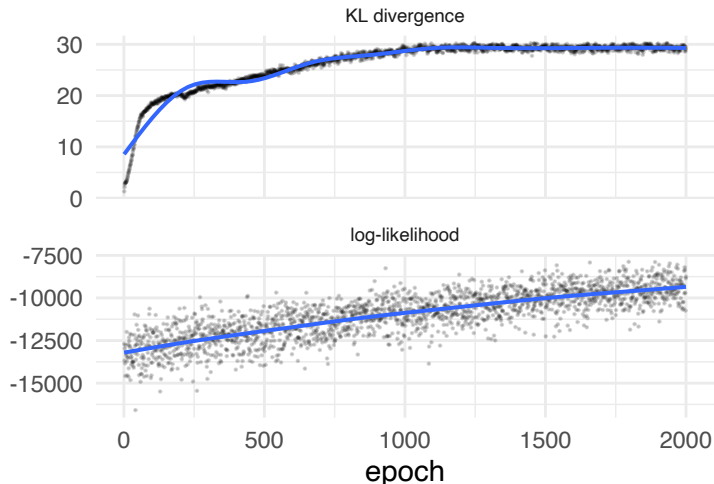
$\beta = 0.1$



- Higher β : lower KL (more Gaussian) but worse reconstruction

Loss traces across epochs

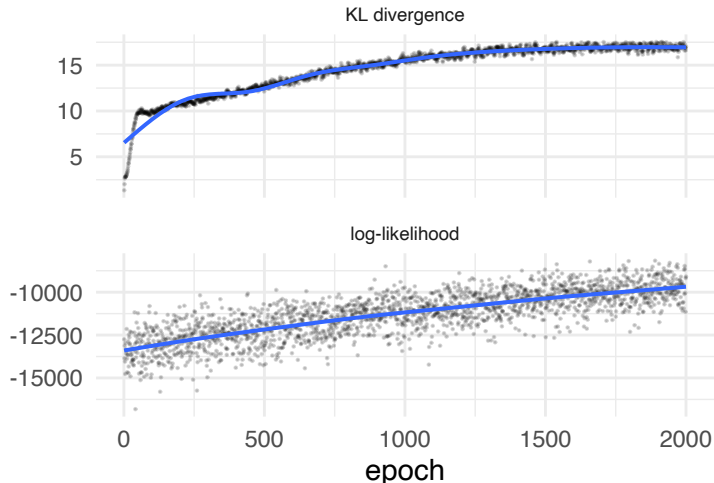
$\beta = 1$



- Higher β : lower KL (more Gaussian) but worse reconstruction

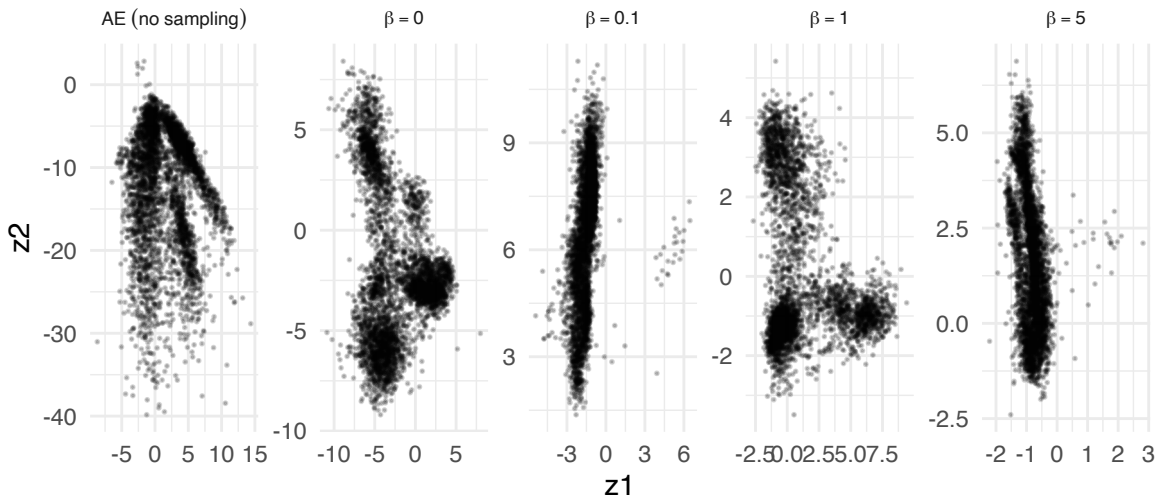
Loss traces across epochs

$\beta = 5$

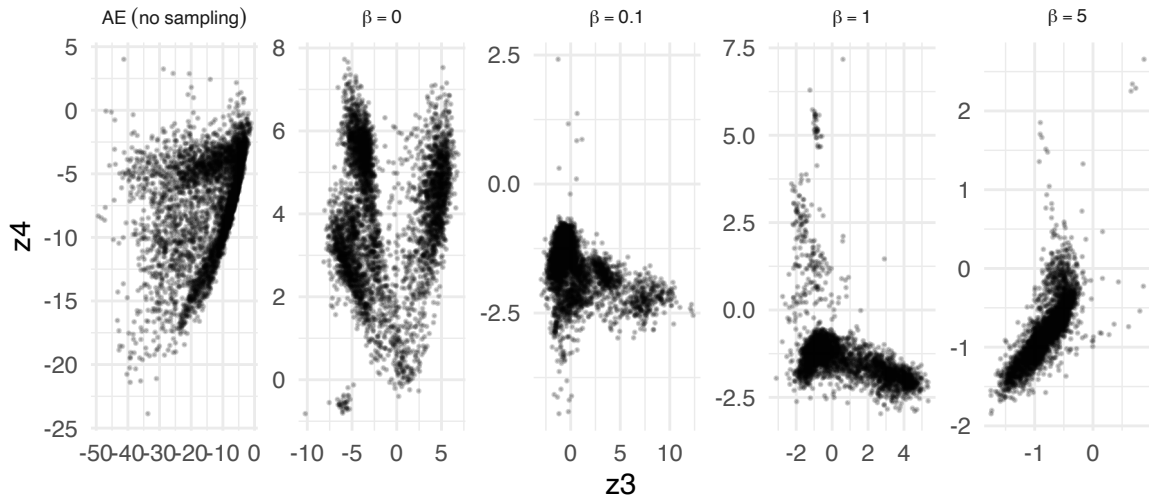


- Higher β : lower KL (more Gaussian) but worse reconstruction

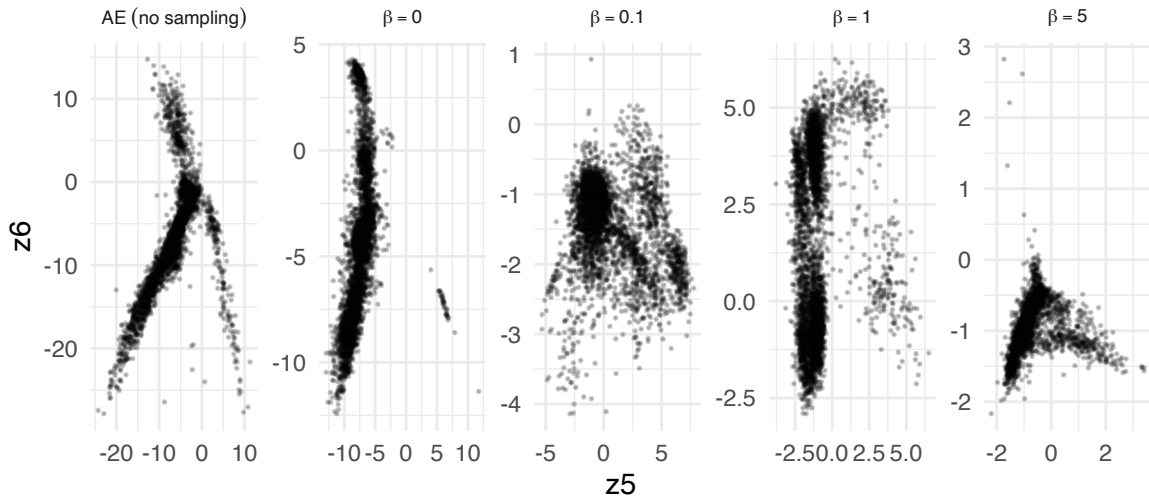
Latent space shape at different β



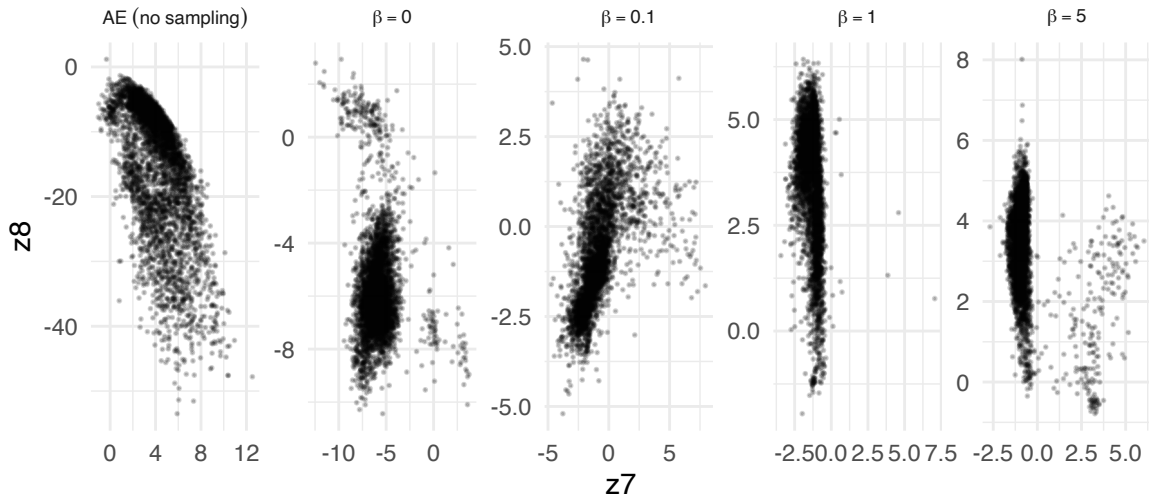
Latent space shape at different β



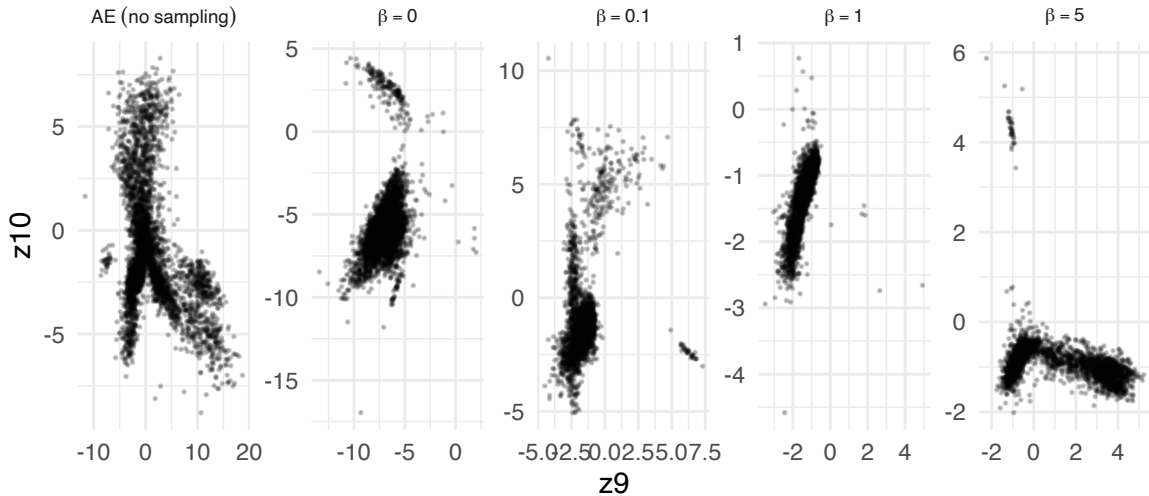
Latent space shape at different β



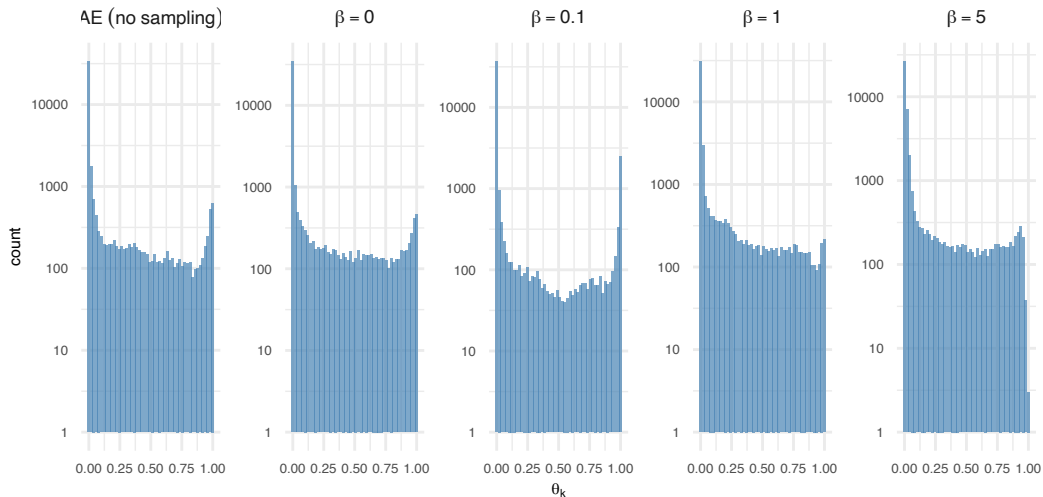
Latent space shape at different β



Latent space shape at different β

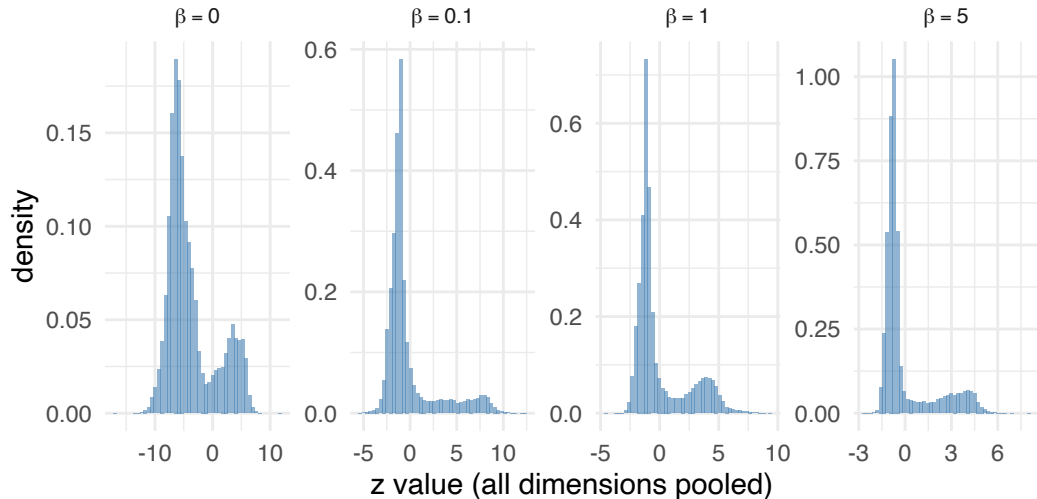


Topic proportions $\theta = \text{softmax}(\mathbf{z})$



- AE: spikes near 0 and 1 — one topic dominates per cell

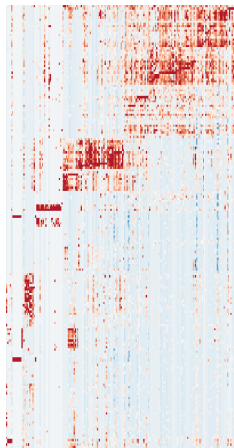
Latent distribution: is z Gaussian?



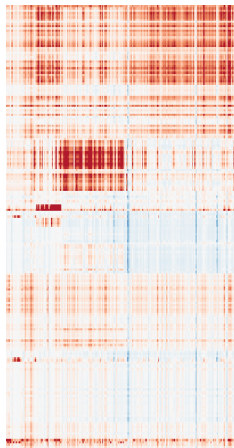
- Higher $\beta \Rightarrow$ closer to the standard normal prior

Reconstruction at different β

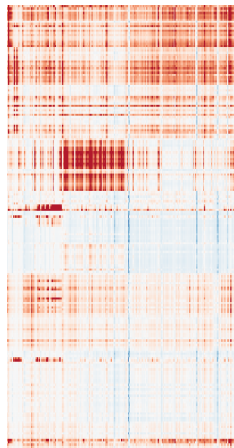
Observed



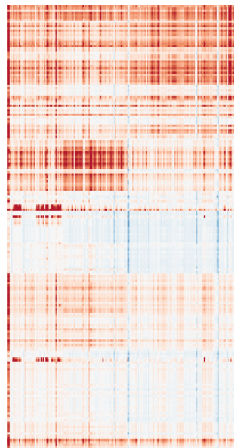
$\beta = 0$



$\beta = 1$

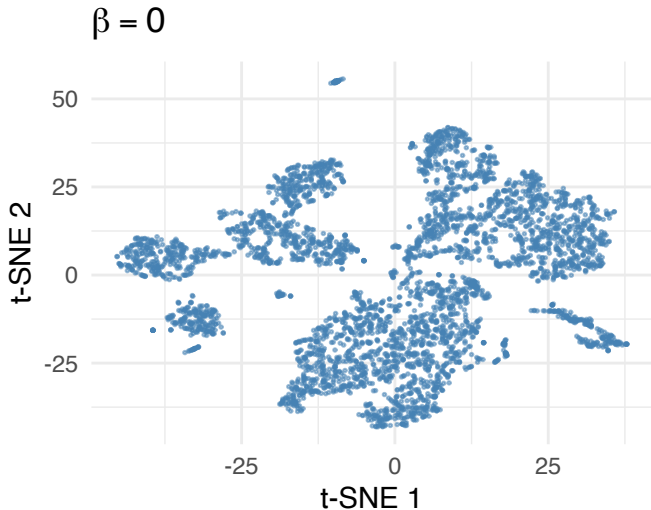


$\beta = 5$



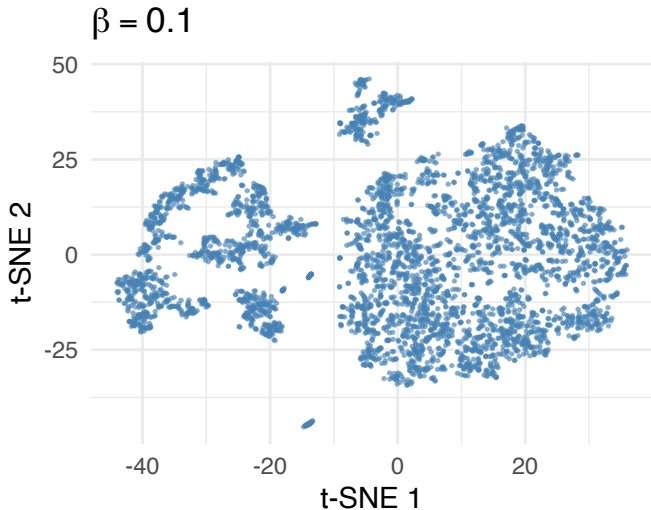
- $\beta = 0$ reconstructs best but latent space is unstructured

Latent space structure at different β



- t-SNE visualization of latent space $\mathbf{z}$

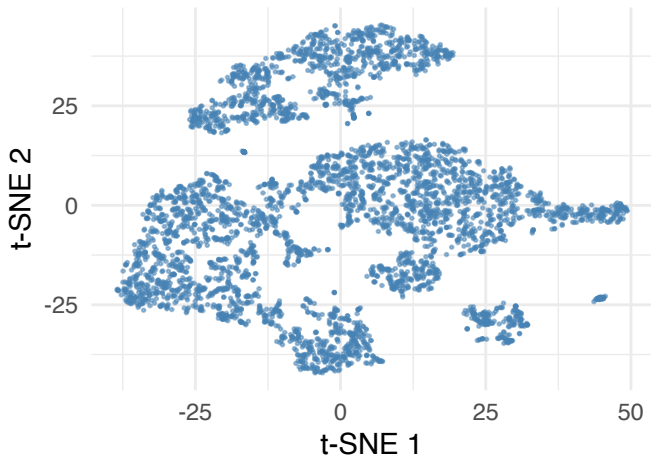
Latent space structure at different β



- t-SNE visualization of latent space $\mathbf{z}$

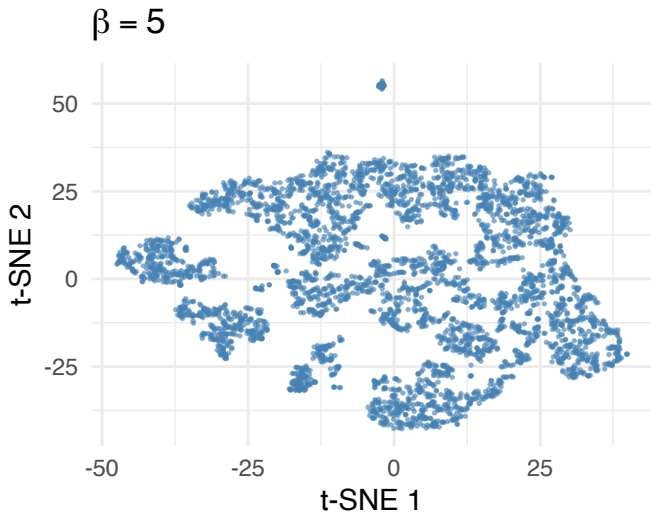
Latent space structure at different β

$\beta = 1$



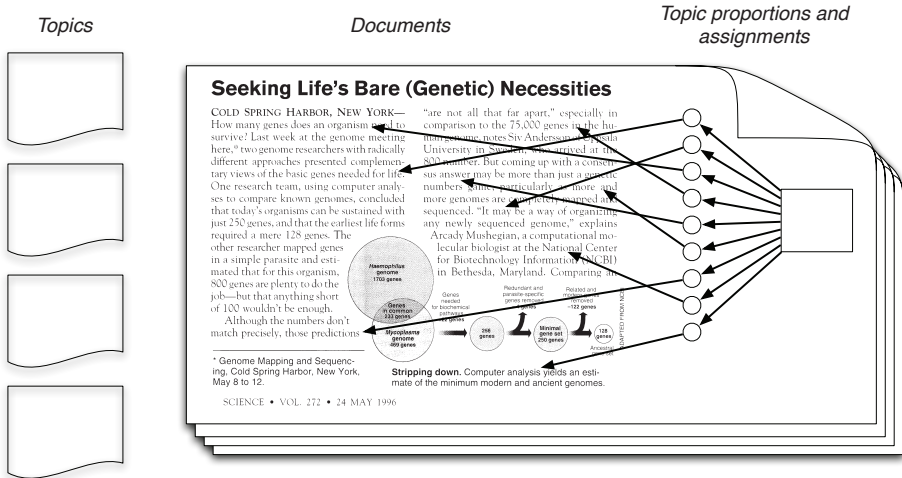
- t-SNE visualization of latent space $\mathbf{z}$

Latent space structure at different β



- t-SNE visualization of latent space $\mathbf{z}$

Connection to topic models



Slide credit: David Blei

Word frequencies define topics

Topics

gene 0.04
dna 0.02
genetic 0.01
...

life 0.02
evolve 0.01
organism 0.01
...

brain 0.04
neuron 0.02
nerve 0.01
...

data 0.02
number 0.02
computer 0.01
...

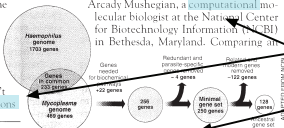
Documents

Seeking Life's Bare (Genetic) Necessities

COLD SPRING HARBOR, NEW YORK—How many **genes** does an **organism** need to **survive**? Last week at the genome meeting here,* two genome researchers with radically different approaches presented complementary views of the basic genes needed for **life**. One research team, using **computer** analyses to compare known **genomes**, concluded that today's **organisms** can be sustained with just 250 genes, and that the earliest life forms required a mere 128 **genes**. The other researcher mapped genes in a simple parasite and estimated that for this organism, 800 genes are plenty to do the job—but that anything short of 100 wouldn't be enough.

Although the numbers don't match precisely, those **predictions**

"are not all that far apart," especially in comparison to the 75,000 **genes** in the human genome, notes Siv Andersson, a biologist at Uppsala University in Sweden, who arrived at the 800 number. But coming up with a consensus answer may be more than just a **genetic** numbers game, particularly as more and more **genomes** are completely mapped and sequenced. "It may be a way of organizing any newly **sequenced genome**," explains Arcady Mushegian, a **computational** molecular biologist at the National Center for Biotechnology Information (NCBI) in Bethesda, Maryland. Comparing an



* Genome Mapping and Sequencing, Cold Spring Harbor, New York, May 8 to 12.

Stripping down. Computer analysis yields an estimate of the minimum modern and ancient genomes.

SCIENCE • VOL. 272 • 24 MAY 1996

Topic proportions and assignments



This is LDVAE (Svensson et al., 2020)

What we just built:

- Linear encoder + multinomial decoder + VAE

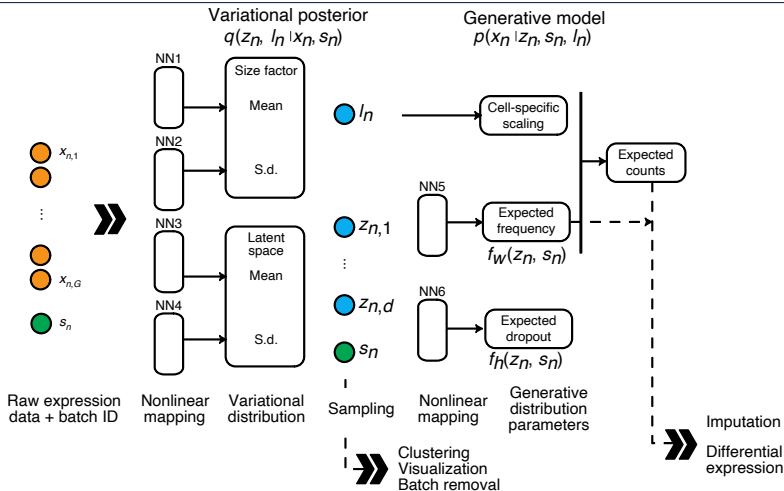
This is exactly **Linearly Decoded VAE** (LDVAE):

- The decoder weight matrix β is directly interpretable
- Each row = a “topic” = a gene program
- Equivalent to probabilistic PCA with count likelihood

scVI (Lopez et al., 2018) extends this with:

- Deep (nonlinear) encoder and decoder
- ZINB likelihood (handles overdispersion + dropout)
- Batch effect correction via conditional decoder

scVI: VAE with more flexible encoder, decoder



(Lopez et al. 2018)

Batch correction with scVI

Problem: PDAC data has 3 tissues (3 batches)

- Technical variation between batches can dominate biological signal
- PCA/UMAP may cluster by batch instead of cell type

scVI solution: Conditional decoder $p(x \mid z, s)$

- Latent code z : biological variation (shared across batches)
- Batch indicator $s \in \{T1, T2, T3\}$: technical variation
- Decoder learns to generate batch-specific expression from shared z

Result: Latent space z is batch-corrected

- Cells group by biology, not batch
- Can integrate multiple datasets/experiments

Implementing scVI in R torch

Strategy: Reuse VAE encoder, upgrade decoder to ZINB with batch conditioning

- Same encoder as before (outputs $\mu, \log \sigma^2, z$)
- New decoder: outputs ZINB parameters (μ, θ, π) conditioned on batch
- Loss = $-\log p(x|z, s) + D_{\text{KL}}(q(z|x)||p(z))$

Why ZINB likelihood?

- Negative binomial handles overdispersion (variance $>$ mean)
- Zero-inflation handles dropout (technical zeros)
- Better fit than multinomial for scRNA-seq counts

ZINB likelihood math

Negative Binomial distribution:

$$\text{NB}(x|\mu, \theta) = \frac{\Gamma(x + \theta)}{\Gamma(\theta) \Gamma(x + 1)} \left(\frac{\theta}{\theta + \mu} \right)^{\theta} \left(\frac{\mu}{\theta + \mu} \right)^x$$

- μ : mean expression
- θ : dispersion (inverse overdispersion)

Zero-Inflated NB:

$$\text{ZINB}(x|\mu, \theta, \pi) = \begin{cases} \pi + (1 - \pi) \cdot \text{NB}(0|\mu, \theta) & \text{if } x = 0 \\ (1 - \pi) \cdot \text{NB}(x|\mu, \theta) & \text{if } x > 0 \end{cases}$$

- π : dropout probability (probability of technical zero)

Why Zero-Inflated Negative Binomial (ZINB)?

Problem with multinomial/Poisson: Cannot handle

- ① **Overdispersion:** variance $\gg$ mean in real scRNA-seq data
- ② **Excess zeros:** both biological (gene off) and technical (dropout)

Solution: ZINB = mixture model with two sources of counts

For gene j in cell i :

$$x_{ij} \sim \begin{cases} 0 & \text{with probability } \pi_{ij} \text{ (dropout)} \\ \text{NB}(\mu_{ij}, \theta_j) & \text{with probability } 1 - \pi_{ij} \end{cases}$$

- μ_{ij} : mean expression (from decoder)
- θ_j : dispersion parameter (gene-specific overdispersion)
- π_{ij} : dropout probability (zero-inflation)

ZINB likelihood in torch - 1

```
## Negative Binomial log-probability
nb.log.prob <- function(x, mu, theta) {
  eps <- 1e-6
  mu <- torch_clamp(mu, min = eps, max = 1e6)
  theta <- torch_clamp(theta, min = eps, max = 1e6)

  log_nb <- (torch_lgamma(x + theta) - torch_lgamma(theta) -
             torch_lgamma(x + 1) +
             theta * torch_log(theta / (theta + mu)) +
             x * torch_log(mu / (theta + mu)))

  log_nb
}
```

$$\text{NB}(x|\mu, \theta) = \frac{\Gamma(x + \theta)}{\Gamma(\theta) \Gamma(x + 1)} \left(\frac{\theta}{\theta + \mu} \right)^{\theta} \left(\frac{\mu}{\theta + \mu} \right)^x$$

- μ : mean expression
- θ : dispersion (inverse overdispersion)

ZINB likelihood in torch - 2

```
## Zero-Inflated NB log-probability
zinb.log.prob <- function(x, mu, theta, pi) {
  eps <- 1e-6
  pi <- torch_clamp(pi, min = eps, max = 1 - eps)

  ## For x = 0: log( + (1-)·NB(0))
  ## For x > 0: log((1-)·NB(x))
  nb_logprob <- nb.log.prob(x, mu, theta)
  zero_nb <- nb.log.prob(torch_zeros_like(x), mu, theta)

  ## Use logaddexp for numerical stability
  log_prob_zero <- torch_logaddexp(
    torch_log(pi + eps),
    torch_log(1 - pi + eps) + zero_nb
  )
  log_prob_nonzero <- torch_log(1 - pi + eps) + nb_logprob

  ## Select based on whether x is zero
  zero_mask <- (x < eps)
  torch_where(zero_mask, log_prob_zero, log_prob_nonzero)
}
```

Putting them all together

```
## Full ZINB log-likelihood (sum over genes, mean over cells)
zinb.llik <- function(x, mu, theta, pi) {
  log_probs <- zinb.log.prob(x, mu, theta, pi)
  ## Clamp to avoid extreme values
  log_probs <- torch_clamp(log_probs, min = -1e6, max = 1e6)
  torch_mean(torch_sum(log_probs, dim = 2))
}
```

Batch-conditional ZINB decoder

```
zinb.decoder <- nn_module(  
  initialize = function(K, d.out, n.batches, h = 128) {  
  
    ## Batch embedding  
    self$batch.embed <- nn_embedding(n.batches, 8)  
  
    ## Hidden layer  
    self$hidden <- nn_linear(K + 8, h)  
    ## Normalization  
    self$bn <- nn_batch_norm1d(h)  
  
    ## Output for ZINB parameters  
    self$mu.head <- nn_linear(h, d.out)  
    self$pi.head <- nn_linear(h, d.out)  
  
    ## Gene-specific dispersion  
    self$log.theta <- nn_parameter(torch_ones(d.out) * log(10))  
  },  
  forward = function(z, batch, lib.size) {  
    ## (...)  
  }  
)
```

- Batch embedding: 3 batches $\rightarrow$ 8-dim vectors
- Concatenate with latent code z before decoder
- Three output heads:
 μ , π
(cell+gene-specific),
 θ (gene-specific)

Batch-conditional ZINB decoder (cont'd)

```
zinb.decoder <- nn_module(  
  ## (...)  
  forward = function(z, batch, lib.size) {  
    batch.emb <- self$batch.embed(batch + 1L)  # 1-indexing  
    ## Concatenate latent code with batch embedding  
    z.batch <- torch_cat(list(z, batch.emb), dim = 2)  
    ## Hidden layer with batch norm + ReLU  
    h <- self$hidden(z.batch) |> self$bn() |> nnf_relu()  
    ## Output ZINB parameters  
    ## : mean expression (scale by library size)  
    mu.raw <- nnf_softplus(self$mu.head(h))  
    ## Normalize library sizes to mean 1 for stability  
    lib.scale <- (lib.size / torch_mean(lib.size))$unsqueeze(2)  
    nb.mu <- mu.raw * lib.scale  
    ## : dropout probability  
    nb.pi <- torch_sigmoid(self$pi.head(h))  
    ## : dispersion (shared across cells, gene-specific)  
    nb.theta <- torch_exp(self$log.theta)$unsqueeze(1)$expand_as(nb.mu)  
    list(nb.mu = nb.mu, nb.theta = nb.theta, nb.pi = nb.pi)  
  }  
)
```

- Batch embedding: 3 batches $\rightarrow$ 8-dim vectors
- Concatenate with latent code z before decoder
- Three output heads:
 μ, π
(cell+gene-specific),
 θ (gene-specific)

Complete scVI model

```
vae.zinb <- nn_module(  
  initialize = function(d.in, K, n.batches) {  
    self$enc <- vae.encoder(d.in, K)          ## REUSE existing VAE encoder  
    self$dec <- zinb.decoder(K, d.in, n.batches) ## NEW: ZINB decoder  
  },  
  forward = function(x, batch, lib.size) {  
    ## Encode  
    q <- self$enc(x) # Returns: z, mu, logvar  
    ## Decode (batch-conditional)  
    params <- self$dec(q$z, batch, lib.size)  
    ## Return everything  
    list(z = q$z, mu = q$mu, logvar = q$logvar,  
         nb.mu = params$nb.mu,  
         nb.theta = params$nb.theta,  
         nb.pi = params$nb.pi)  
  }  
)
```

- Only decoder changes – encoder is identical to previous VAE!
- Decoder now takes batch labels and library sizes as input

Train scVI with ZINB loss

```
## Initialize
```

```
model.scvi <- vae.zinb(n.genes, K, n.batches)
optimizer <- optim_adam(model.scvi$parameters, lr = 1e-3)
```

```
## Training loop
```

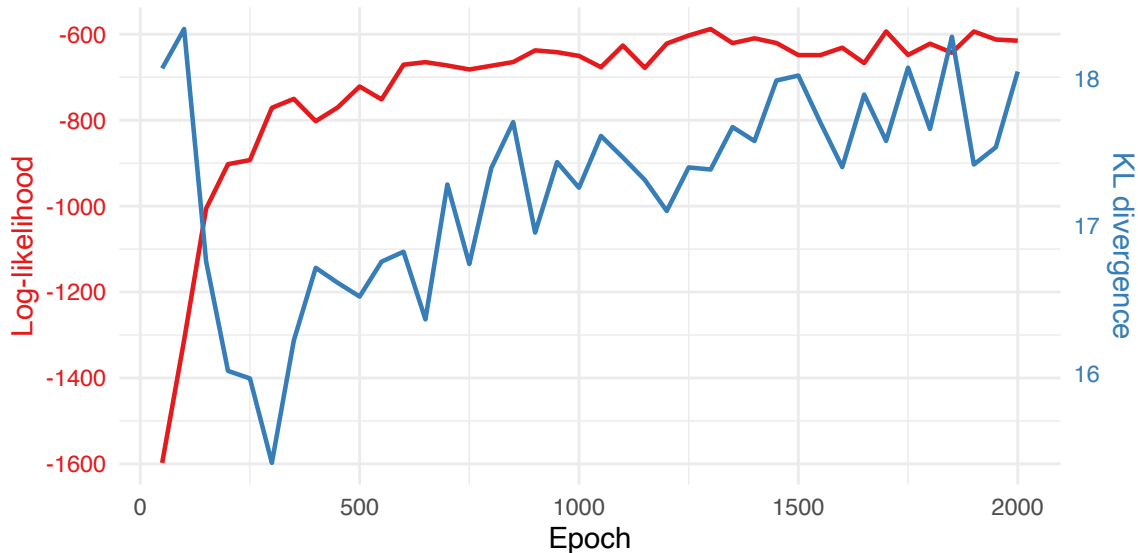
```
for (epoch in seq_len(2000)) {
  idx <- sample(n.cells, 256)
  x.batch <- X.tensor[idx, ]
  batch.batch <- batch.tensor[idx]
  lib.batch <- lib.size.tensor[idx]

  out <- model.scvi(x.batch, batch.batch, lib.batch)

  ## Loss = -ZINB log-likelihood + KL
  llik <- zinb.llik(x.batch, out$nb.mu, out$nb.theta, out$nb.pi)
  kl <- kl.loss(out$mu, out$logvar)
  loss <- -llik + kl

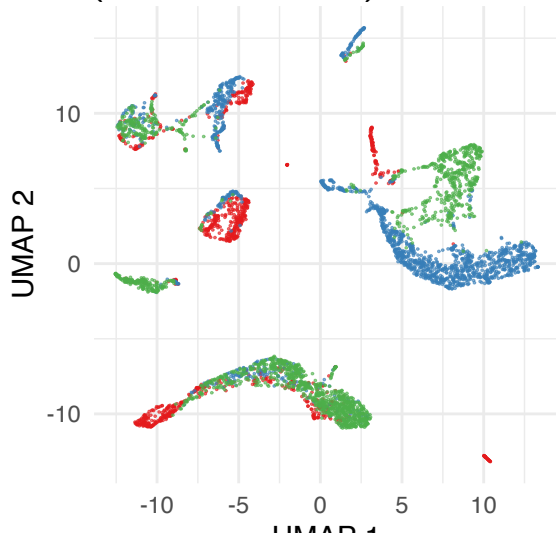
  optimizer$zero_grad()
  loss$backward()
  optimizer$step()
}
```

scVI training converges smoothly



Batch correction comparison

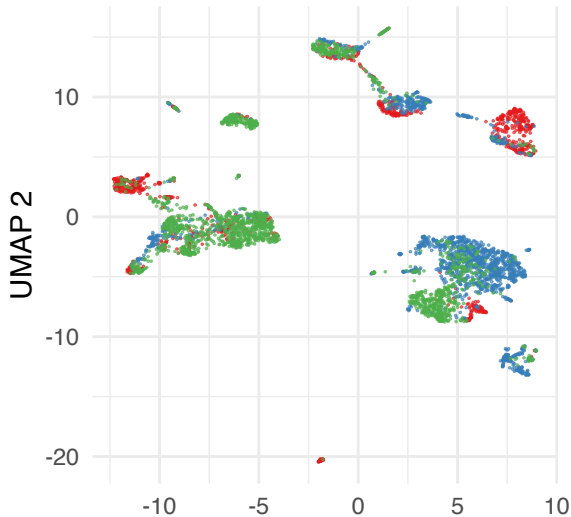
PCA (batch-confounded)



Yongjin Park

Variational Autoencoder in Single-cell Data

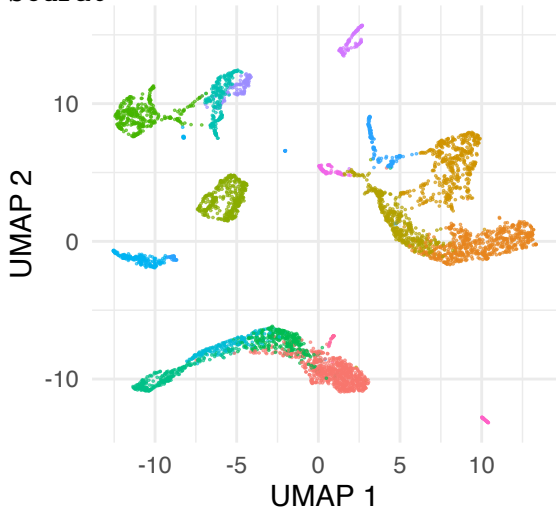
scVI (batch-corrected)



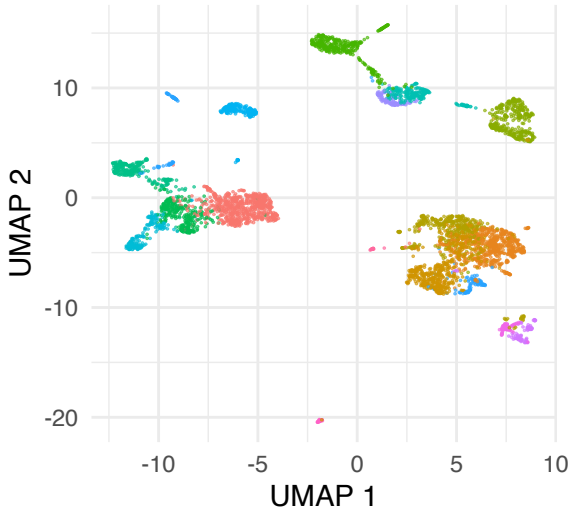
10 March 2026

Seurat PCA clustering $\approx$ scVI

Seurat



scVI



Quantifying batch mixing

Batch entropy metric:

- For each cell, find $k = 30$ nearest neighbors
- Compute entropy of batch labels among neighbors
- Higher entropy = better batch mixing (tissues)

Method	Batch Entropy
PCA	0.319
scVI	0.410

- scVI increases batch mixing by 28.5%

What did we learn?

scVI in R torch shows:

- ① **Modular design:** Reuse VAE encoder, only change decoder + likelihood
- ② **ZINB likelihood:** Three parameters (μ , θ , π) handle overdispersion + dropout
- ③ **Batch conditioning:** Simple embedding layer enables integration
- ④ **Interpretable:** See exactly what each component does

Key takeaways:

- From multinomial $\rightarrow$ ZINB: better fit to scRNA-seq count data
- Conditional decoder: batch correction without destroying biology
- Build-it-yourself: understand what's “under the hood” of scvi-tools

Running scVI with Python scvi-tools

```
import scvi
import scanpy as sc
import numpy as np

# Load data (convert from Seurat to AnnData)
# In practice: use sceasy or manual conversion
adata = sc.read_h5ad("pdac_tissues123.h5ad")
# Setup scVI model
scvi.model.SCVI.setup_anndata(
    adata,
    layer="counts",           # raw counts
    batch_key="orig.ident"   # tissue batch (T1, T2, T3)
)
# Create and train model
model = scvi.model.SCVI(
    adata,
    n_latent=10,              # K = 10 latent dimensions
    gene_likelihood="zinb"    # zero-inflated negative binomial
)

model.train(max_epochs=100)
```

Extract latent representation

```
# Get latent representation (batch-corrected)
adata.obsm["X_scVI"] = model.get_latent_representation()

# Compute UMAP on scVI latent space
sc.pp.neighbors(adata, use_rep="X_scVI")
sc.tl.umap(adata)

# Compare: PCA vs scVI
sc.pl.umap(adata, color=["orig.ident", "seurat_clusters"])
```

Key difference from LDVAE:

- Deep encoder/decoder (nonlinear)
- ZINB likelihood handles dropout
- Batch correction via conditional generation

References

- Higgins, I, L Matthey, Arka Pal, et al. 2016. “Beta-VAE: Learning Basic Visual Concepts with a Constrained Variational Framework.” *Int Conf Learn Represent*, November.
- Kharchenko, Peter V. 2021. “The Triumphs and Limitations of Computational Methods for scRNA-Seq.” *Nat. Methods*, June.
- Lopez, Romain, Jeffrey Regier, Michael B Cole, Michael I Jordan, and Nir Yosef. 2018. “Deep Generative Modeling for Single-Cell Transcriptomics.” *Nat. Methods* 15 (12): 1053–58.